

컨볼루션을 활용한 일반화

8장에서 다루는 내용

- 컨볼루션 이해
- 컨볼루션 신경망 만들기
- 커스텀 nn.Module 서브클래스 만들기
- 모듈과 함수형 API의 차이
- 신경망 설계시 고려할 점

앞서 7장에서는 선형 계층에 존재하는 최적화를 위해 활용 가능한 여러 파라미터의 도움으로 데이터에 맞춘(혹은 과적합된) 단순 신경망을 만들었다. 다만 모델은 새나 비행기의 특성을 일반화하여 훈련하기보다는 훈련셋을 암기하는 성향을 보였다. 모델의 아키텍처를 살피면서 왜 이런 상황이 발생하는지에 관해 추측도 해봤다. 완전 연결로 설정하면 이미지 안의 새나 비행기의 모든 가능한 위치를 발견해야 하므로, 너무 많은 파라미터가 필요하고(그 덕분에 모델이 훈련셋을 암기해버리기 쉽게 만든다) 위치에 독립적이지도 못했다(즉 일반화하기 어려웠다). 7장에서 다룬 바와 같이, 일반화를 강제하기 위해 훈련셋 이미지를 이리저리 잘라내어 훈련 데이터를 증강시켜도 여전히 너무 많은 파라미터가 필요했다.

그래서 더 나은 방법을 알려주고자 한다! 신경망에서 완전 연결된 아핀 변환 유닛을 또 다른 선형 연산인 컨볼루션으로 바꾸는 것이다.

8.1 컨볼루션

이번에는 컨볼루션을 깊게 이해해보고 신경망에서 어떻게 사용할 수 있는지를 배워보자. 지금은 새와 비행기를 구별해야 하는 상황이고 친구는 우리의 해결 방법을 기다리고 있지만, 잠시 컨볼루션을 배우는 데 시간을 쓸 가치는 충분하다. 컴퓨터 비전의 기본 개념에 대한 직관을 키우고 단단히 무장해서 다시 돌아와 문제를 풀자.

이 절에서는 컨볼루션이 어떻게 인접한 정보를 사용하고 위치 변화에 대한 불변성을 획득하는지를 배워보겠다. 컨볼루션을 정의하는 공식을 자세히 살펴보고, 펜과 종이를 사용해서 적용해보자. 핵심은 공식 대신 그림에서 볼 수 있을 테니 너무 겁먹지는 말자.

앞에서 `nn.Linear`를 사용해 입력 이미지를 1차원으로 생각하고 `n_output_features × n_input_features`라는 가중치 행렬을 곱했다. 때문에 이미지의 각 채널은 가중치 집합과 곱해진 모든 픽셀의 총합을 각각의 출력 피처에 대해 계산했다.

하늘을 나는 비행기 같은 물체와 일치하는 패턴을 인지하려면 인근 픽셀의 배열 방식을 살펴봐야 하며, 서로 멀리 떨어져 있는 픽셀들을 연결해가며 어떻게 보이는지 확인하는 데에는 관심이 없다. 즉 스피트파이어가 등장하는 이미지에 나무나 구름 혹은 연 같은 물체가 구석에 있는지는 관심이 없다는 뜻이다.

이 개념을 수학적인 형태로 바꾸려면 이미지 내의 다른 픽셀이 아닌 바로 옆 픽셀에 대한 가중치의 합을 계산하면 된다. 출력 피쳐와 출력 픽셀 위치마다 가중치 행렬을 만드는 것인데, 센터가 되는 픽셀에서 일정 거리 이상 떨어진 경우 가중치가 0이 된다고 생각하면 된다. 계산 결과는 여전히 가중치의 합이므로 선형 연산으로 볼 수 있다.

8.1.1 컨볼루션의 역할

앞서 우리는 바람직한 속성 하나를 파악한 바 있다. 바로, 지역화된 패턴이 이미지의 어떤 위치에 있더라도 동일하게 출력에 영향을 주는 성질을 **평행이동 불변성**translation invariance이라 했다. 7장에서 이미지를 벡터처럼 다루려고 만들었던 행렬로 이런 효과를 내려면 다소 복잡한 가중치 패턴을 구현해야 한다(너무 복잡하지 않으니 걱정하지 말자. 잠시 후에 개선될 것이다). 가중치 행렬의 대부분은 0이 될 것이고(출력 픽셀로부터 멀리 떨어진 입력 픽셀이라도 영향을 줄 수 있게 한다), 입력에 대해 상대적으로 같은 위치에 있는 엔트리가 출력 픽셀에 동일한 영향을 줄 방법을 찾아야 한다. 바꿔 말하면, 같은 값으로 초기화해야 하며 그래서 이 경우와 관련된 가중치가 훈련을 통해 업데이트시 영향을 받지 않아야 한다는 의미다. 이런 식으로 지역 패턴에 대응하여 가중치는 반응하고, 해당 패턴이 이미지의 어디에 존재하든지 파악할 수 있다.

여기서 설명한 방법은 실행 불가능하지 않다. 심지어 이미지에 대해 지역적인, 평행이동 불변성을 보장하는 선형 연산도 이미 존재하는데, 이것이 바로 **컨볼루션**convolution이다. 컨볼루션에 대해서는 더 간결하게 설명할 수 있지만 우리는 방금 기술한 것처럼 조금은 다른 각도에서 자세히 설명하려 한다.

컨볼루션, 아니 조금 정확하게 말하자면 **이산 컨볼루션**discrete convolution¹(연속된 버전도 있지만 여기에서는 다루지 않으려 한다)은 2차원 이미지에 가중치 행렬을 스칼라곱을 수행하는 것으로 정의한다. 가중치 행렬은 **커널**kernel이라 부르며 입력의 모든 이웃에 대해 수행한다. 2차원 텐서인 3×3 커널을 생각해보자(딥러닝에서는 통상 크기가 작은 커널을 사용하는데 그 이유는 나중에 살펴보자).

```
weight = torch.tensor([[w00, w01, w02],  
                        [w10, w11, w12],  
                        [w20, w21, w22]])
```

그리고 다음과 같은 1채널의 $M \times N$ 이미지가 있다고 하자.

```
image = torch.tensor([[i00, i01, i02, i03, ..., i0N],  
                     [i10, i11, i12, i13, ..., i1N],  
                     [i20, i21, i22, i23, ..., i2N],  
                     [i30, i31, i32, i33, ..., i3N],  
                     ...  
                     [iM0, iM1, iM2, iM3, ..., iMN]])
```

그러면 편향값 없는 출력 이미지의 각 요소 계산은 다음과 같다.

```
o11 = i11 * w00 + i12 * w01 + i13 * w02 +  
      i21 * w10 + i22 * w11 + i23 * w12 +  
      i31 * w20 + i32 * w21 + i33 * w22
```

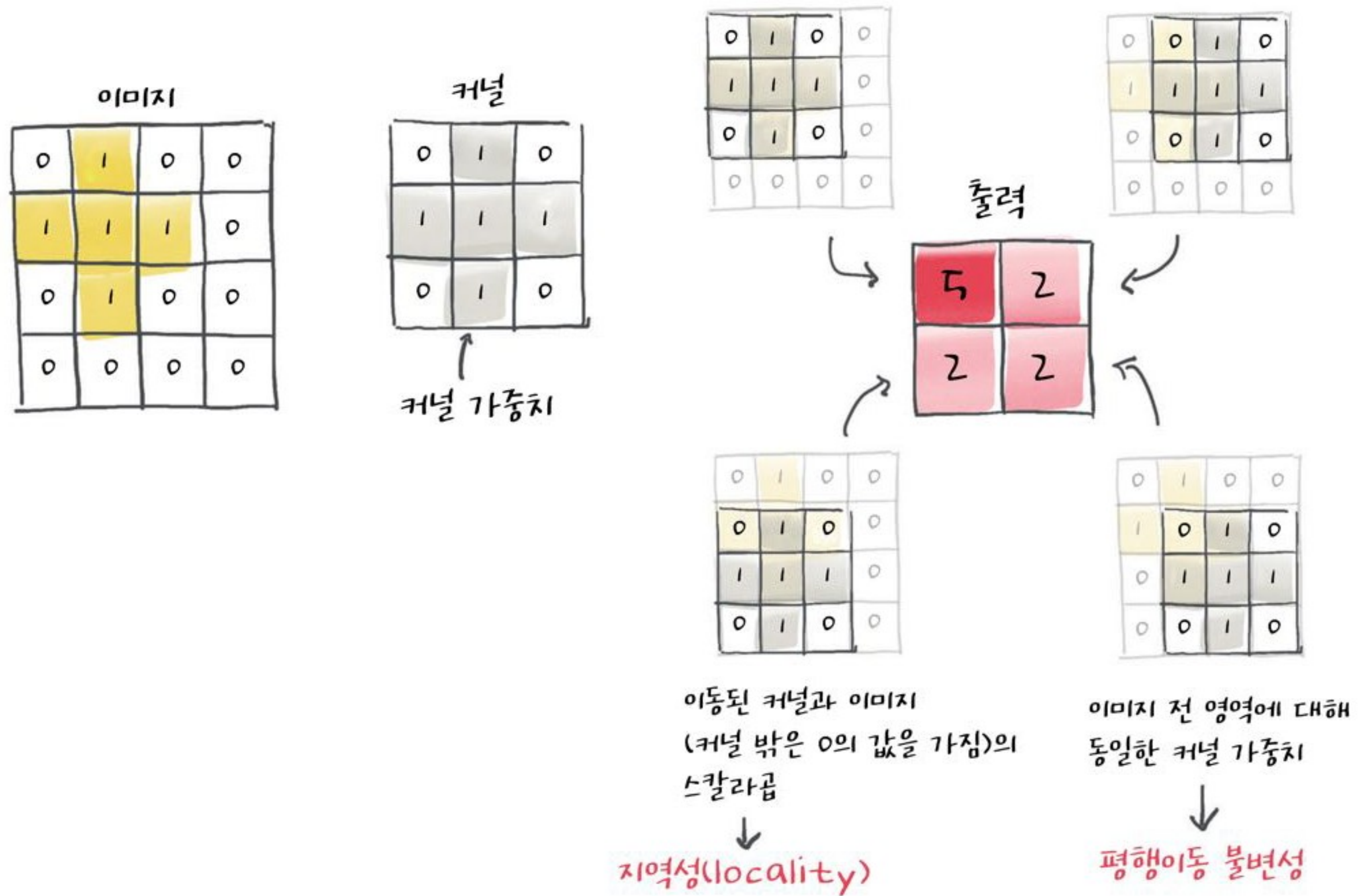



그림 8.1 컨볼루션: 지역성과 평행이동 불변성

커널을 입력 이미지의 $i11$ 으로 평행이동했고 이 위치의 입력 이미지 값과 가중치를 곱했다. 이런 식으로 입력의 모든 위치에 대해 커널을 이동시켜 가면서 가중치의 합을 구해 출력 이미지를 만들었다. RGB 이미지처럼 채널이 여러 개인 경우 가중치 행렬은 $3 \times 3 \times 3$ 행렬로 각 채널에 대한 가중치 집합이 존재하며, 이를 합쳐서 출력값 계산에 기여하게 된다.

`nn.Linear`에서의 `weight` 행렬의 요소와 같이 커널의 가중치는 미리 알 수 없다. 랜덤으로 초기화되고 역전파를 통해 업데이트된다. 동일한 커널의 각 가중치가 전체 이미지에 대해 계속 사용된다는 점도 기억하자. 자동미분을 생각하면 각 가중치가 전체 이미지를 훑은 결과라고 볼 수 있다. 이렇게 하기 때문에 컨볼루션 가중치에 대한 손실값의 미분은 이미지 전체가 기여한다고 볼 수 있다.

이제 앞에서 언급한 내용과의 연관성을 파악할 수 있을 것이다. 컨볼루션은 어떤 한 픽셀 주위를 제외한 나머지의 가중치를 0으로 하는 여러 개의 선형 연산이며 훈련 중에 각각으로부터 동일한 업데이트를 받는다.

이렇게 컨볼루션을 사용하면,

- 주위 영역에 대한 지역 연산을 할 수 있고,
- 평행이동 불변성을 가지며,
- 더 적은 파라미터를 사용한다.

더 적은 파라미터만 쓰는 이유는 이전의 완전 연결 계층과는 달리 컨볼루션에서의 파라미터 수는 이미지 픽셀 수에 의존하지 않기 때문이다. 대신 컨볼루션 커널의 크기(3×3 , 5×5 ... 식으로)와 모델에서 얼마나 많은 컨볼루션 필터(혹은 출력 채널 수)를 쓰는지에 의존한다.

8.2 컨볼루션 사용해보기

복잡한 이야기는 충분히 했으니, 이제 새와 비행기를 구분하는 도전에서 컨볼루션을 파이토치로 어떻게 사용하는지를 알아보자. `torch.nn` 모듈은 1, 2, 3차원에 대한 컨볼루션을 제공한다. `nn.Conv1d`는 시계열용이고 `nn.Conv2d`는 이미지용이며 `nn.Conv3d`는 용적 데이터나 동영상용이다.

CIFAR-10 데이터에 대해서는 `nn.Conv2d`를 사용한다. `nn.Conv2d`에 전달하는 인자는 최소 입력 피쳐 수(픽셀 하나당 둘 이상의 값을 가지는 멀티채널 이미지의 경우 채널 정보)와 출력 피쳐 수, 커널의 크기가 있어야 한다. 우리의 첫 컨볼루션 모듈은 RGB 채널을 가지니 픽셀당 3개의 입력 피쳐를 가지고 출력 피쳐는 임의로 16을 전달하자. 출력 이미지가 더 많은 채널을 가질수록 신경망의 용량도 커진다. 현재는 각 채널이 다양한 종류의 피쳐를 발견하기 원한다. 또 이들을 랜덤으로 초기화하기 때문에 훈련 후에 어떤 피쳐는 필요없어질 수 있다.² 일단 커널 크기는 3×3 으로 정하자.

커널의 크기는 일반적으로 모든 방향으로 동일하게 만든다. 그래서 편의상 `kernel_size=3`으로 지정하면 파이토치는 2차원 컨볼루션에 대해 3×3 (파이썬에서 (3, 3) 튜플로 주는 것처럼)으로 인식한다. 3차원 컨볼루션에서는 $3 \times 3 \times 3$ 으로 인식한다. 이 책의 2부에서 살펴볼 CT 스캔 데이터는 세 개의 축 중에 하나가 다른 복셀^{Voxel}(3D 화소)을 가진다. 이 경우 커널이 예외적인 차원에 대해 다른 크기를 가지는 것이 맞다. 하지만 지금은 모든 차원에 대해 동일한 크기를 가지는 컨볼루션에 집중하자.

```
# In[11]:
```

```
conv = nn.Conv2d(3, 16, kernel_size=3)
```

```
conv
```

← `kernel_size`에 3 대신 아래의 출력 결과처럼
`kernel_size=(3, 3)`을 전달해도 동일한 효과가 있다.

```
# Out[11]:
```

```
Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1))
```


weight 텐서는 어떤 차원 정보를 가질까? 커널은 3×3 이므로 가중치 역시 3×3 을 사용한다. 출력 픽셀 하나에 대해 커널은 입력 채널이 $in_ch = 3$ 이므로 출력 픽셀 값 하나에 대해(출력 채널 전체에 대한 평행이동 불변성을 가지면서) $in_ch \times 3 \times 3$ 이 된다. 그리고 이 값을 출력 채널만큼 가지게 되어 $out_ch = 16$ 이므로 전체 가중치 텐서는 $out_ch \times in_cn \times 3 \times 3$, 즉 $16 \times 3 \times 3 \times 3$ 이 된다. 편향값의 크기는(설명을 위해 편향값을 언급하지 않았지만 선형 모듈의 경우와 같이 출력 이미지의 각 채널에 더하는 상수 값으로 생각하면 된다) 16이다. 이 가정이 맞는지 확인해보자.

```
# In[12]:  
conv.weight.shape, conv.bias.shape  
  
# Out[12]:  
(torch.Size([16, 3, 3, 3]), torch.Size([16]))
```


이미지 학습에 컨볼루션이 얼마나 유용한지 확인할 수 있다. 이제 전체 이미지를 돌며 해당 가중치를 최적화하기 위해 부분 패턴을 찾는 훨씬 더 작은 모델을 만들었다.

2차원 컨볼루션 전달은 2차원 이미지를 출력한다. 여기서 출력 픽셀은 입력 이미지의 각 이웃 데이터에 대한 가중치의 합이다. 우리의 경우는 커널 가중치와 편향값 `conv.weight`가 랜덤으로 초기화된다. 따라서 출력 이미지 자체가 특별한 의미를 갖지 않는다. 하나의 입력 이미지로 `conv` 모듈을 호출하려면 `nn.Conv2d`는 입력으로 $B \times C \times H \times W$ 를 받으므로 통상 0번째 차원을 `unsqueeze`를 통해 배치 차원으로 사용한다.

```
# In[13]:  
img, _ = cifar2[0]  
output = conv(img.unsqueeze(0))  
img.unsqueeze(0).shape, output.shape
```

```
# Out[13]:  
(torch.Size([1, 3, 32, 32]), torch.Size([1, 16, 30, 30]))
```

그래도 궁금하니 어떤 모양인지 그림 8.2에 출력해봤다.

```
# In[15]:  
plt.imshow(output[0, 0].detach(), cmap='gray')  
plt.show()
```

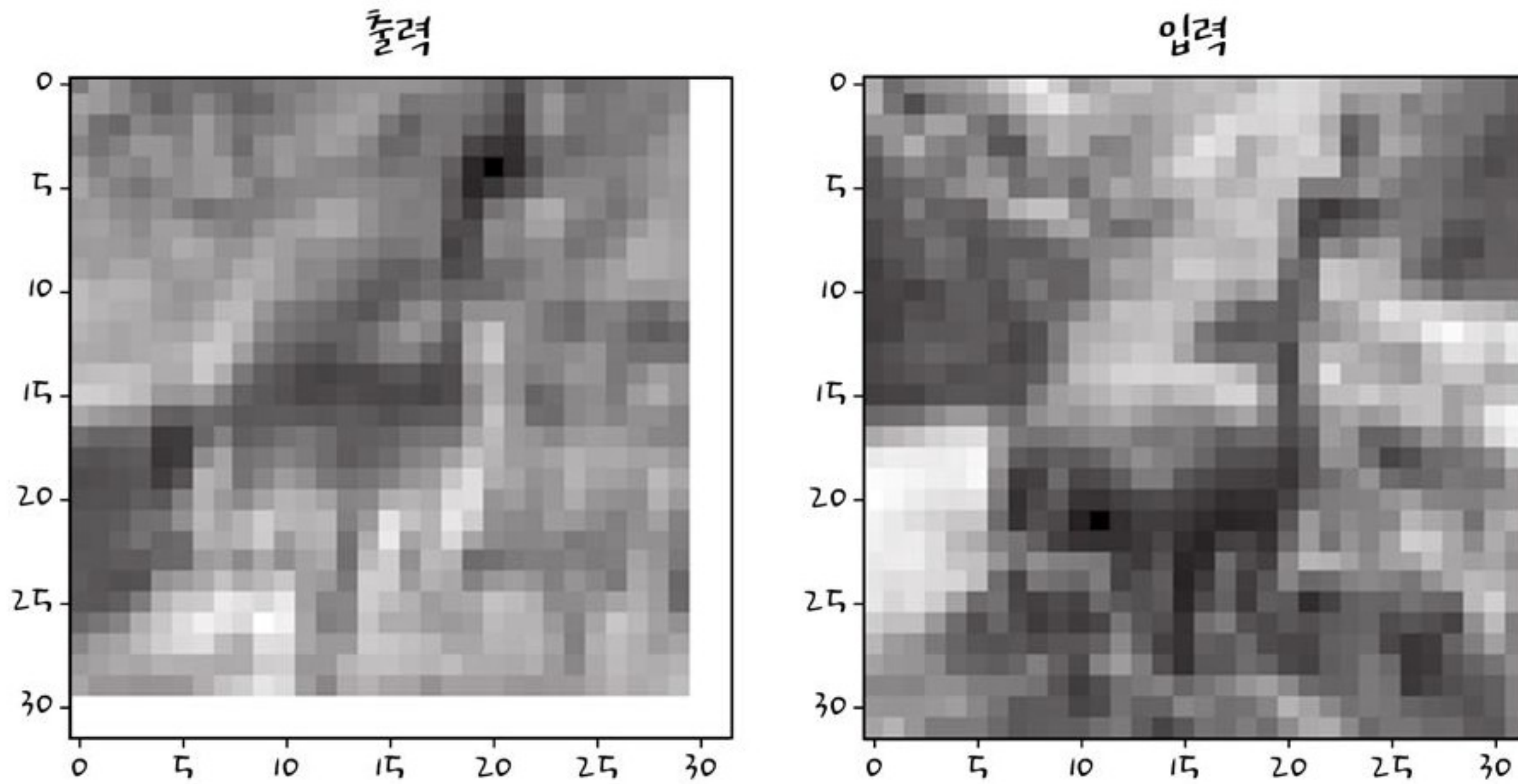


그림 8.2 랜덤 컨볼루션을 통과한 새 그림(입력도 보여주려고 코드를 살짝 건드렸다)

잠깐만. output 크기를 보자. `torch.Size([1, 16, 30, 30])`이다. 음? 픽셀이 조금 잘린 것 같다. 무슨 일일까?

8.2.1 경계 패딩하기

출력 이미지가 입력 이미지보다 작은 것은 이미지의 경계에서 이뤄지는 작업에 따른 부작용이다. 3×3 이웃 영역에 대해 컨볼루션 커널을 가중치합으로 적용하려면 일단 모든 방향에 값이 존재한다는 가정이 있어야 한다. $i00$ 지점에서 보면 픽셀은 오른쪽과 아래쪽에만 있다. 기본적으로 파이토치는 컨볼루션 커널을 입력 그림 안에서 미끌어지며 이동하는데, 수평과 수직 위치는 $\text{width} - \text{kernel_width} + 1$ 로 구한다. 따라서 크기가 홀수인 컨볼루션 커널의 길이(우리의 경우 $3//2 = 1$)의 절반만큼 양쪽의 그림이 잘린다. 이 때문에 각 차원에서 딱 두 픽셀만큼 없어진 것이다.

다행히 파이토치는 컨볼루션과 관련해 이미지의 경계에 값이 0인 **가짜ghost** 픽셀을 패딩해주는 기능을 제공한다. 그림 8.3에서 어떻게 패딩하는지를 보여준다.

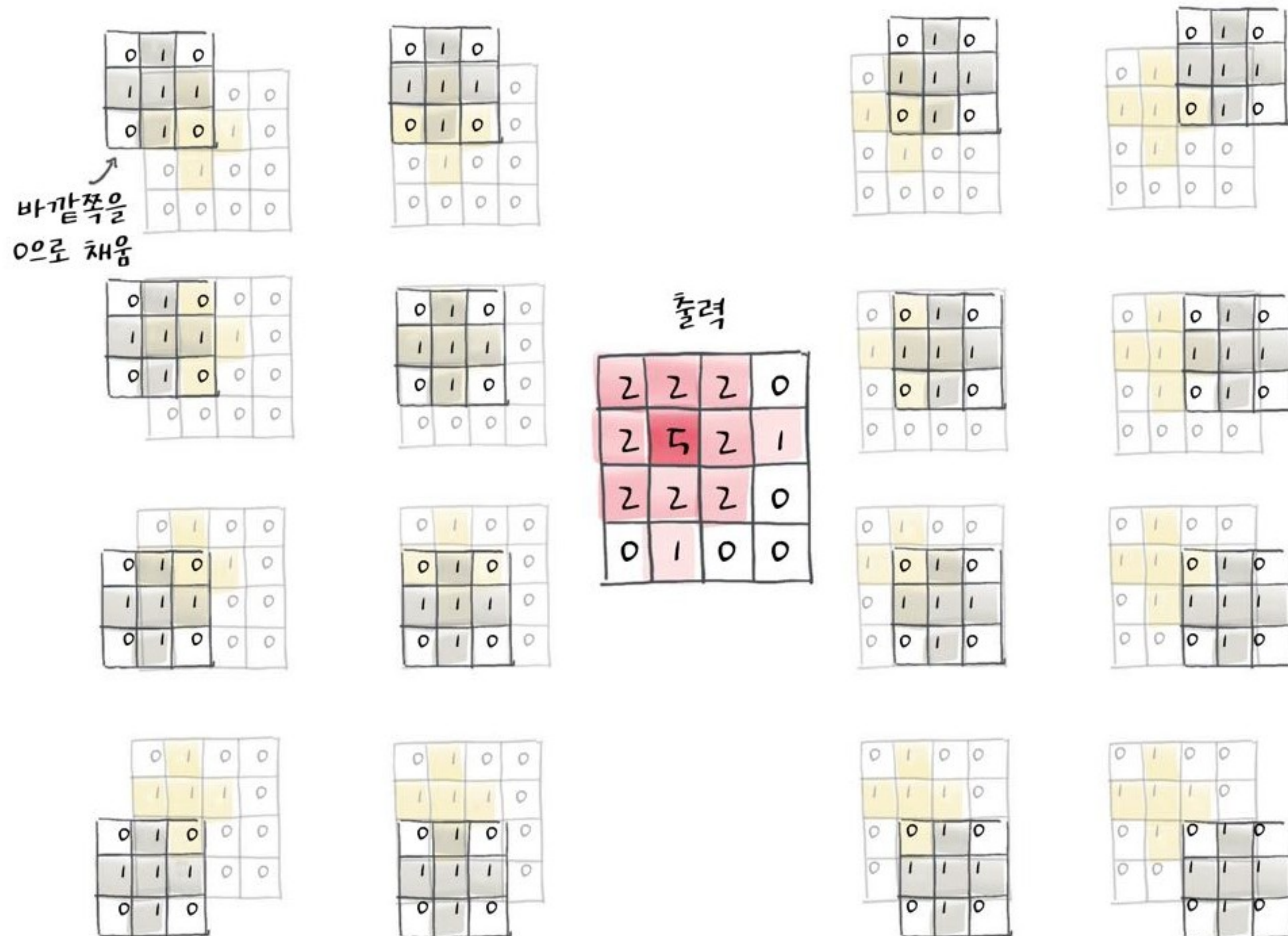


그림 8.3 출력 이미지 크기를 보존하기 위한 제로 패딩

우리의 경우 `kernel_size=3`일 때 `padding=1`로 지정하면 i00 지점에서 왼쪽과 위쪽에 추가 이웃이 만들어진다. 그래서 원래 이미지에서 모서리에 해당하는 지점에서도 컨볼루션의 출력값을 계산할 수 있다.³ 덕분에, 출력이 입력과 동일한 크기를 가진다.

```
# In[16]:
```

```
conv = nn.Conv2d(3, 1, kernel_size=3, padding=1) ← 패딩시킴
output = conv(img.unsqueeze(0))
img.unsqueeze(0).shape, output.shape
```

```
# Out[16]:
```

```
(torch.Size([1, 3, 32, 32]), torch.Size([1, 1, 32, 32]))
```


패딩 사용 여부와 상관없이 weight과 bias의 크기는 변하지 않는 점을 기억하자.

컨볼루션에서 패딩을 사용하는 이유는 크게 두 가지다. 하나는 패딩을 통해서 컨볼루션과 이미지의 크기 변경 문제를 별도로 분리해 기억해야 하는 사항을 하나 줄이는 데 도움을 준다. 다른 하나는 이를 통해 컨볼루션 구조 자체에 더 신경쓸 수 있다는 점이다. 8.5.3절에서 다룰 스킵 커넥션skip connection이나 2부에서 다룰 유넷U-Net 같은 구조를 예로 들 수 있는데 몇 개의 컨볼루션 앞뒤의 텐서를 호환되는 크기로 만들면 쉽게 더하거나 뺄 수 있게 된다.

8.2.2 컨볼루션으로 피쳐 찾아내기

앞에서 weight이나 bias는 역전파를 통해 학습되는 파라미터라고 배웠다. 정확하게는 nn.Linear에서 일어난다. 하지만 이와 다르게 직접 가중치를 설정해서 컨볼루션에서 어떤 일이 일어나는지 알아볼 수도 있다.

먼저 bias를 0으로 제거해 교란 변수를 배제하자. 그리고 가중치에 상수값을 넣어서 출력에서의 각 픽셀이 자신의 이웃 픽셀에 대한 평균을 가지게 해보자. 3×3 이웃에 대한 코드는 다음과 같다.

```
# In[17]:  
with torch.no_grad():  
    conv.bias.zero_()  
  
with torch.no_grad():  
    conv.weight.fill_(1.0 / 9.0)
```

여기에서 conv.weight.one_()을 사용해서 출력값이 이웃 픽셀값의 합이 되게 할 수도 있다. 이렇게 하면 출력 이미지의 값이 9배 커지는 점 외에는 차이가 없다.

아무튼 이제 CIFAR 이미지에 미친 영향을 확인해보자.

```
# In[18]:  
output = conv(img.unsqueeze(0))  
plt.imshow(output[0, 0].detach(), cmap='gray')  
plt.show()
```

예상할 수 있다시피 그림 8.4에서처럼 이미지가 흐려졌다. 각 출력 픽셀은 자신의 주변 픽셀에 대한 평균이기 때문에 출력 픽셀에서 이러한 상관관계를 반영해 픽셀 간의 변화가 부드러워졌다.

다음으로는 커널을 조금 바꿔보자. 처음에는 다음 커널이 조금 이상해 보일 수 있다.


```
# In[19]:
conv = nn.Conv2d(3, 1, kernel_size=3, padding=1)

with torch.no_grad():
    conv.weight[:] = torch.tensor([[-1.0, 0.0, 1.0],
                                    [-1.0, 0.0, 1.0],
                                    [-1.0, 0.0, 1.0]])

    conv.bias.zero_()
```

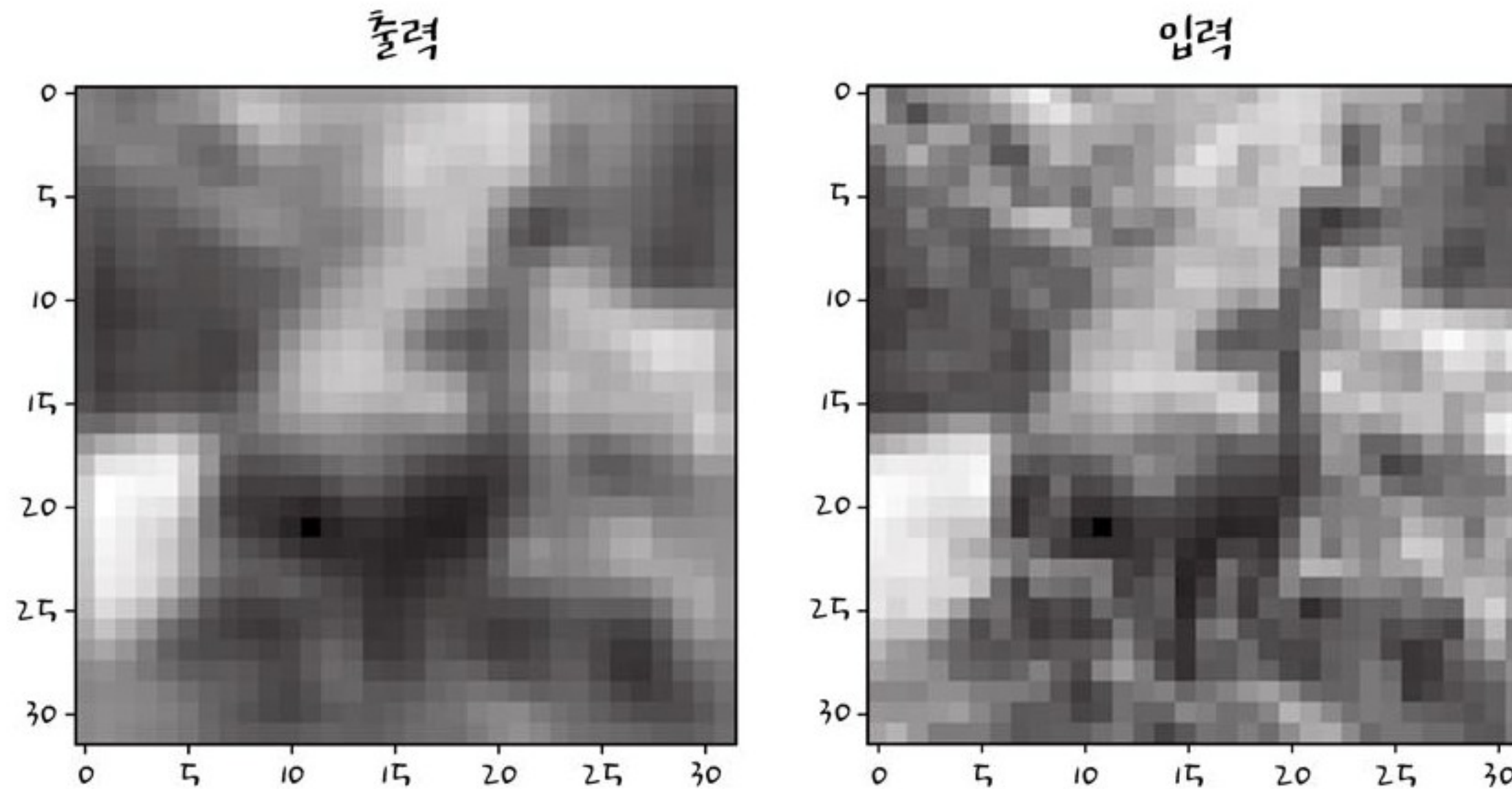


그림 8.4 상수 컨볼루션 커널 덕에 우리 새가 흐려졌다.

앞에서 일반 컨볼루션 커널로 했던 것처럼 아무 위치나 잡아 가중치의 합을 구한다고 치면 위치 2,2에 대해 다음 값을 얻는다.

$$\begin{aligned} o_{22} = & i_{13} - i_{11} + \\ & i_{23} - i_{21} + \\ & i_{33} - i_{31} \end{aligned}$$

이 커널은 i_{22} 의 오른쪽 픽셀에서 i_{22} 의 왼쪽 픽셀을 빼는 계산을 수행한다. 만일 이미지에서 강도가 다른 두 인접한 영역 사이에서 수직 경계에 커널이 적용되면, o_{22} 는 높은 값을 가질 것이다. 반면 커널이 거의 비슷한 색이나 밀도로 이뤄진 영역에서 수행되면 o_{22} 는 0이 된다. 즉 이 커널은 가로로 인접한 두 영역 사이의 수직 경계를 탐색하는 역할을 한다.

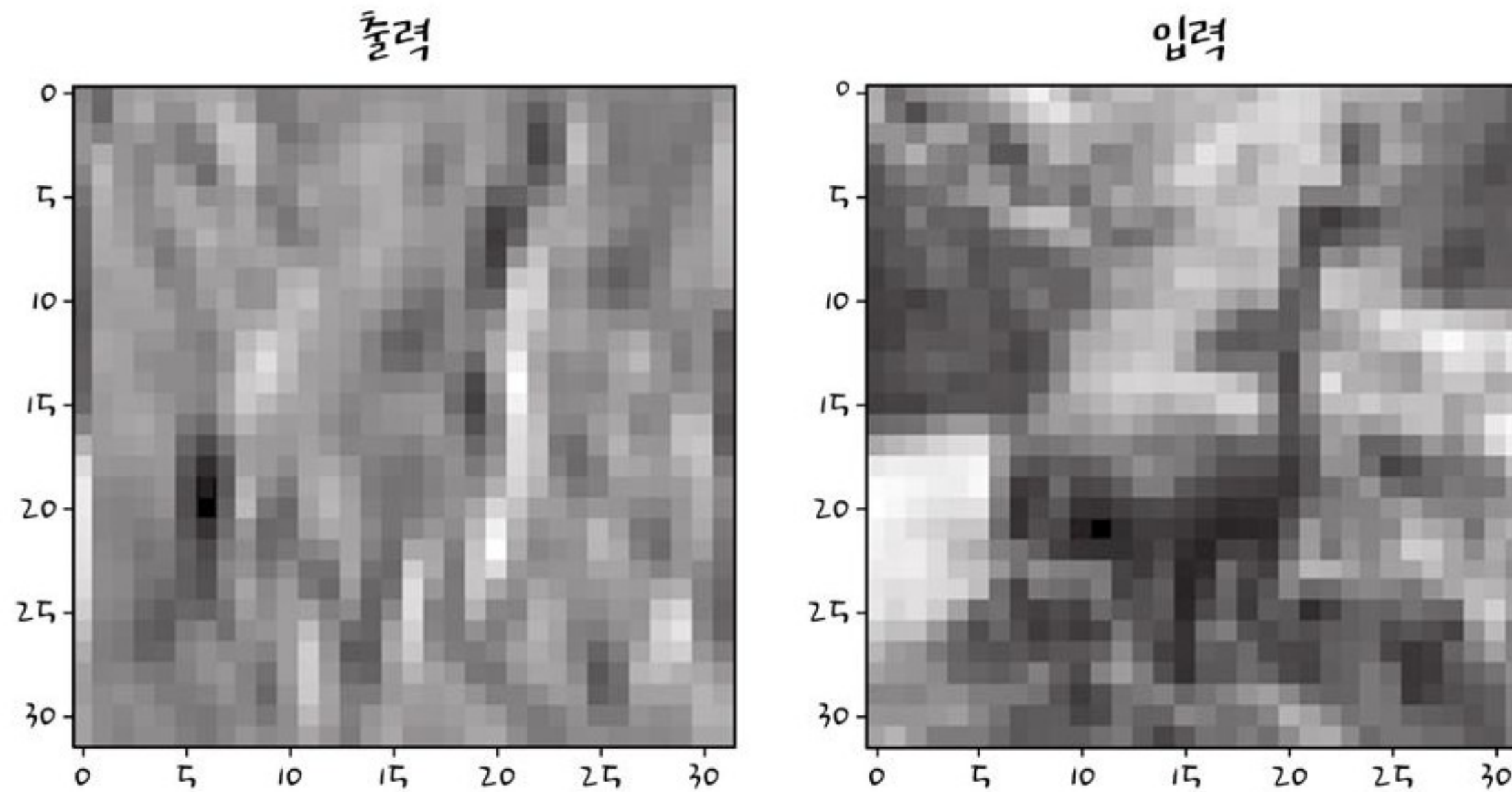


그림 8.5 수작업으로 직접 만든 컨볼루션 커널에 의한 새 이미지의 수직 경계선 처리

이 컨볼루션 커널을 이미지에 적용한 결과를 그림 8.5에서 볼 수 있다. 다들 예상했겠지만, 이 컨볼루션 커널은 수직 경계를 강조한다. 이 외에도 수평이나 대각선 경계 혹은 십자 모양이나 체스판 패턴 등을 탐지하는 훨씬 정교한 필터를 더 만들 수 있다. 여기에서 탐지한다는 의미는 높은 출력값을 가진다는 의미다. 전통적으로 필터 설계는 컴퓨터 비전 전문가의 역할로 이러한 필터를 최적으로 조합하여 이미지의 특징을 강조하는 식으로 물체를 인식해왔다.

반면 이제는 어떻게 해야 가장 효과적으로 구별할 수 있는지는 몰라도, 딥러닝을 통해 데이터로부터 커널을 자동으로 만들게 된다. 예를 들면, 7.2.5절에서 소개한 관측값과 출력값 사이의 음의 크로스엔트로피 손실을 최소화할 수 있다. 이 관점에서 컨볼루션 신경망의 역할은 어떤 멀티 채널 이미지를 다른 멀티 채널 이미지로 변환하는 연속된 계층인 필터 더미 집합인 커널을 추정하는 것이며, 여기서 각 채널은 피처에 대응하게 된다(한 채널은 평균을 다른 채널은 수직 경계선을 탐지하는 경우를 생각해보자). 그림 8.6은 훈련을 통해 어떻게 자동으로 커널을 학습하는지를 보여준다.

8.2.3 깊이와 풀링으로 한 단계 더 인식하기

지금까지는 매우 좋다. 하지만 개념적으로 모두가 알고 있으면서 함구하는 문제가 하나 있다. 완전 연결 계층에서 컨볼루션으로 넘어오면서 지역성이나 평행이동 불변성을 해결했다. 그리고 3×3 이나 5×5 같은 작은 커널을 사용하길 추천했다. 이 정도 크기가 지역성의 한계다. 만약 이미지 내 그림이 꽤 크다면 어떻게 해야 할까? 이미지 안의 물체나 구조가 3픽셀이나 5픽셀밖에 안 될까? 우리는 알 수 없다. 모든 물체가 3~5픽셀 정도로 작지는 않기 때문이다. 그렇다면 더 큰 범위에서 신경망이 패턴을 인식하게 하려면 어떻게 해야 할까? 새와 비행기를 효과적으로 구별하는 해법을 만들려면 이에 대한 해결책이 필요하다. CIFAR-10 이미지는 작지만 물체의 (날개) 길이는 여러 픽셀에 걸쳐 있기 때문이다.

가능한 방법 중 하나는 더 큰 컨볼루션 커널을 사용하는 것이다. 32×32 이미지에 대해 32×32 커널까지는 만들 수 있다. 하지만 이러면 결국 예전에 완전 연결된 아핀 변환으로 수렴해서 컨볼루션의 장점을 잃어버리게 된다. 컨볼루션 신경망에서는 컨볼루션을 차례로 층층이 쌓으면서 동시에 연속적인 컨볼루션 사이의 이미지를 다운샘플링하는 또 다른 방법이 있다.

■ 큰 이미지에서 작은 이미지로: 다운샘플링

다운샘플링은 여러 방법으로 수행할 수 있다. 이미지를 반으로 줄이는 것은 이웃하는 네 개의 픽셀을 입력받아 한 픽셀을 출력하는 작업과 동일하다. 네 개의 입력값에 대해 어떤 값 하나를 출력할 것인지는 우리가 정할 수 있다.

- 네 개의 픽셀을 평균하기: 초기에는 이와 같은 **평균 풀링** average pooling이라는 방법이 일반적이었는데 요즘은 잘 사용하지 않는다.
- 네 개의 픽셀 중 최댓값: **맥스 풀링** max pooling이라 불리는 이와 같은 접근 방법은 오늘날 가장 널리 사용되지만, 데이터의 4분의 3을 버린다는 단점이 있다.
- 스트라이드하며 컨볼루션을 수행하되 n번째 픽셀만 계산하기: 3×4 컨볼루션을 스트라이드 2로 실행하면 이전 계층의 모든 픽셀을 포함한다. 이 방식의 가능성을 보여주는 논문이 있지만 아직 맥스 풀링을 대체할 정도는 아니다.

일단 그림 8.7의 맥스 풀링을 사용해보자. 이 그림에서는 겹치지 않는 2×2 크기의 타일을 사용하며 각 타일에서 최댓값을 뽑아 축소된 비율의 새로운 픽셀로 지정한다.

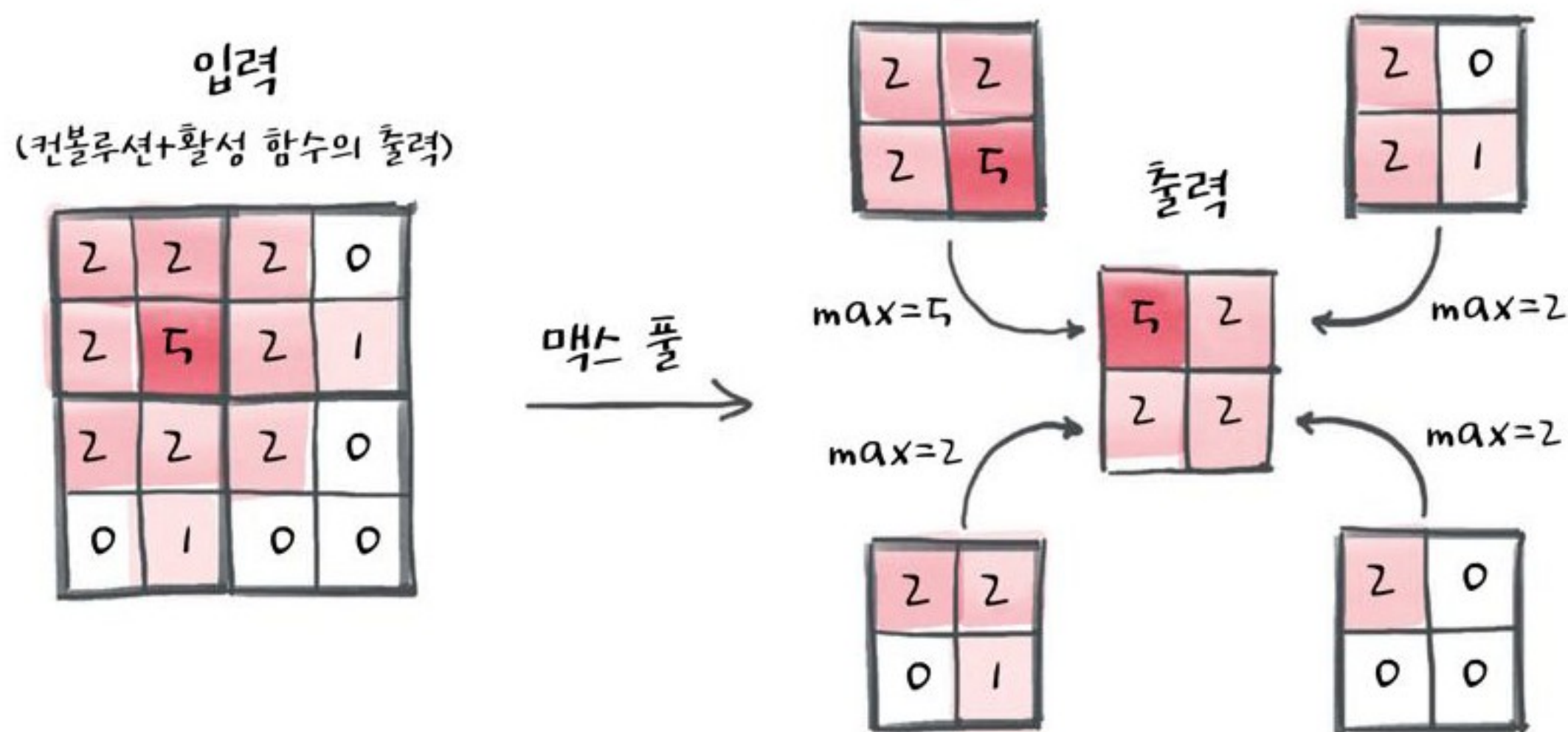


그림 8.7 맥스 풀링의 세부 동작 원리

직관적으로 생각해보면 다른 선형 계층처럼 활성 함수로 연결되므로 컨볼루션층의 출력 이미지의 각 값은 (수직선처럼) 특정 커널에 대응하는 패턴이 발견될 때 높은 값을 가진다. 다운샘플링을 통해 2×2 인접 픽셀에서 최댓값을 뽑는 것은 약한 신호는 버리고 살아남은 피처를 발견하는 과정으로 볼 수 있다.

맥스 풀링은 `nn.MaxPool2d` 모듈에 있다(컨볼루션과 마찬가지로 1차원과 3차원 데이터를 위한 버전도 있다). 입력으로 풀링 연산을 수행할 인접한 영역 크기를 받는다. 이미지를 절반으로 줄이고 싶다면 크기를 2로 지정한다. 입력 이미지에 대해 기대한 대로 동작하는지 확인하자.

```
# In[21]:
pool = nn.MaxPool2d(2)
output = pool(img.unsqueeze(0))

img.unsqueeze(0).shape, output.shape

# Out[21]:
(torch.Size([1, 3, 32, 32]), torch.Size([1, 3, 16, 16]))
```


■ 더 나은 성능을 위해 컨볼루션과 다운샘플링 결합하기

이제 컨볼루션과 다운샘플링의 조합이 어떻게 큰 패턴을 식별해내는지 살펴보자. 그림 8.8에서 우리는 8×8 이미지에 대해 3×3 커널을 적용하여 같은 크기의 멀티 채널 출력 이미지를 얻는다. 그리고 출력 이미지를 반으로 줄여 4×4 이미지를 얻고 다른 3×3 커널을 적용한다. 두 번째 커널셋은 반으로 줄어든 이미지 안에서 3×3 의 인접 픽셀에 대해 동작하므로, 원래 입력의 8×8 인접 영역에 대해 효과적으로 동작하는 셈이다. 두 번째 커널셋은 첫 번째 커널셋의 피쳐 (평균이나 엣지 등의 피쳐)를 받아 추가적인 피쳐를 추출한다.

그래서 첫 번째 커널셋이 먼저 작은 영역 내에서 1차로 저수준의 피처에 대해 동작하는 한편, 두 번째 커널셋은 이렇게 얻은 피처들을 합성한 피처를 만들고 좀 더 넓은 범위에 대해 효과적으로 동작한다. 이런 메커니즘은 매우 강력한데 CIFAR-10 데이터셋에서 얻은 32×32 이미지보다 훨씬 더 복잡한 장면을 인식하는 컨볼루션 신경망을 제공한다.

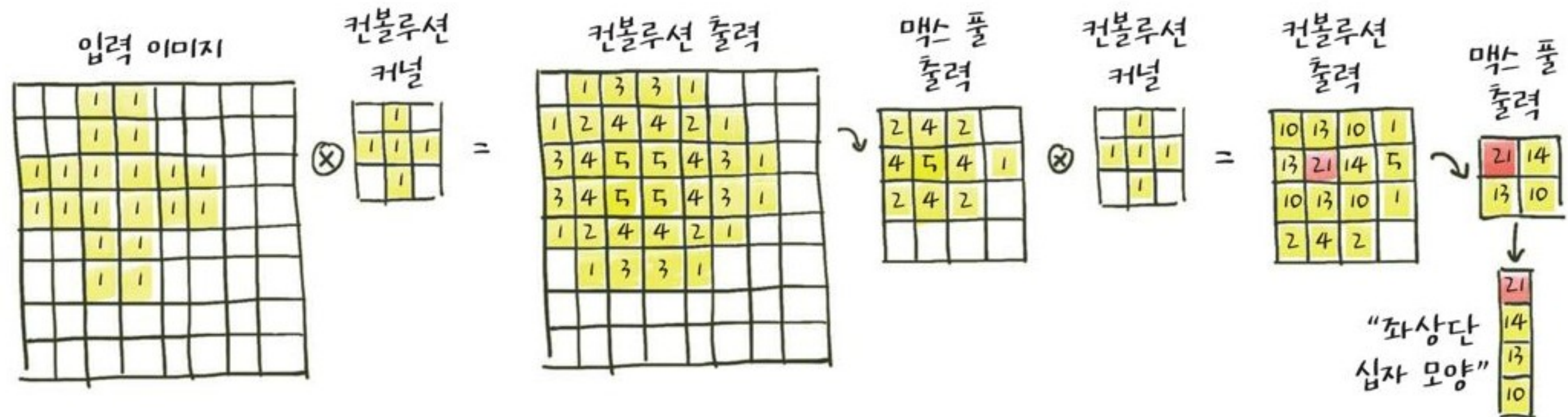


그림 8.8 컨볼루션을 층층이 쌓고 다운샘플링한 효과를 보여주기 위해 직접 그려본 컨볼루션: 큰 십자 모양이 두 개의 작은 십자 모양의 커널과 맥스 풀링을 통해 하이라이트된다.

출력 픽셀의 수용 필드

그림 8.8에서 두 번째 3×3 컨볼루션 커널이 컨볼루션 출력으로 값 21을 출력하는 것은 첫 번째 맥스풀 출력의 좌상단 3×3 픽셀 영역에 대한 값이다. 이 픽셀 영역을 다시 따라 올라가면 첫 번째 컨볼루션 출력의 좌상단 6×6 픽셀 영역과 일치한다. 그리고 이 영역은 다시 첫 번째 컨볼루션이 좌상단의 7×7 픽셀 영역을 계산한 값이다. 따라서 두 번째 컨볼루션의 픽셀은 입력 데이터의 7×7 영역으로부터 영향을 받은 결과다. 첫 번째 컨볼루션은 암묵적으로 패딩 열과 행을 사용하여 모서리에서의 출력값을 계산했다. 이렇지 않았다면 두 번째 컨볼루션 출력의 (경계로부터 떨어진) 픽셀에 대해 입력 픽셀의 8×8 영역이 대응되었을 것이다. 전문 용어로는 3×3 컨볼루션, 2×2 맥스풀, 3×3 컨볼루션 구조의 출력 뉴런은 8×8 의 수용 필드를 가진다고 표현한다.

8.2.4 우리의 신경망에 적용하기

손에 들고 있는 이 빌딩 블록들을 활용하여 새와 비행기를 탐지하는 컨볼루션 신경망을 만들어 보자. 이전의 완전 연결 모델에서 시작해 앞서 설명한 `nn.Conv2d`와 `nn.MaxPool2d`를 활용해본다.

```
# In[22]:  
model = nn.Sequential(  
    nn.Conv2d(3, 16, kernel_size=3, padding=1),  
    nn.Tanh(),  
    nn.MaxPool2d(2),  
    nn.Conv2d(16, 8, kernel_size=3, padding=1),  
    nn.Tanh(),  
    nn.MaxPool2d(2),  
    # ...  
)
```

첫 번째 컨볼루션은 3 RGB 채널을 16, 즉 16개의 독립적인 피처를 만들어 새와 비행기에 대한 저수준의 피처를 찾아낸다. 그리고 Tanh 활성화 함수를 적용한다. 결과로 만들어진 16채널의 32×32 이미지는 이제 첫 MaxPool2d를 통해 16채널의 16×16 이미지로 다운샘플링되고, 8채널 16×16 출력을 만드는 다른 컨볼루션으로 들어간다. 이 출력은 이제 좀 더 높은 수준의 피처를 가질 것이다. 그리고 이어서 Tanh 활성화 함수와 8채널 8×8 출력을 위한 풀링을 수행한다.

어디가 끝인가? 입력 이미지가 8×8 피처 세트로 줄어들고 난 후 이 신경망은 결국 음의 로그 가능도로 넣을 수 있는 확률값을 뽑아줘야 한다. 확률은 1차원 벡터인 숫자 쌍으로 나와야 하는데(하나는 비행기, 다른 하나는 새) 지금은 멀티 채널의 2차원 피처인 상태다.

이 장의 처음으로 돌아가 생각해보면 우리는 이미 뭘 해야 할지 알고 있다. 8채널의 8×8 이미지를 1차원 벡터로 바꿔서 완전 연결 계층으로 신경망을 마무리해야 한다.

```
# In[23]:
model = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1),
    nn.Tanh(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 8, kernel_size=3, padding=1),
    nn.Tanh(),
    nn.MaxPool2d(2),
    # ... ← 경고: 뭔가 중요한 게 하나 빠졌다!
    nn.Linear(8 * 8 * 8, 32),
    nn.Tanh(),
    nn.Linear(32, 2))
```

이는 다음 그림 8.9의 신경망을 구현한 코드다.

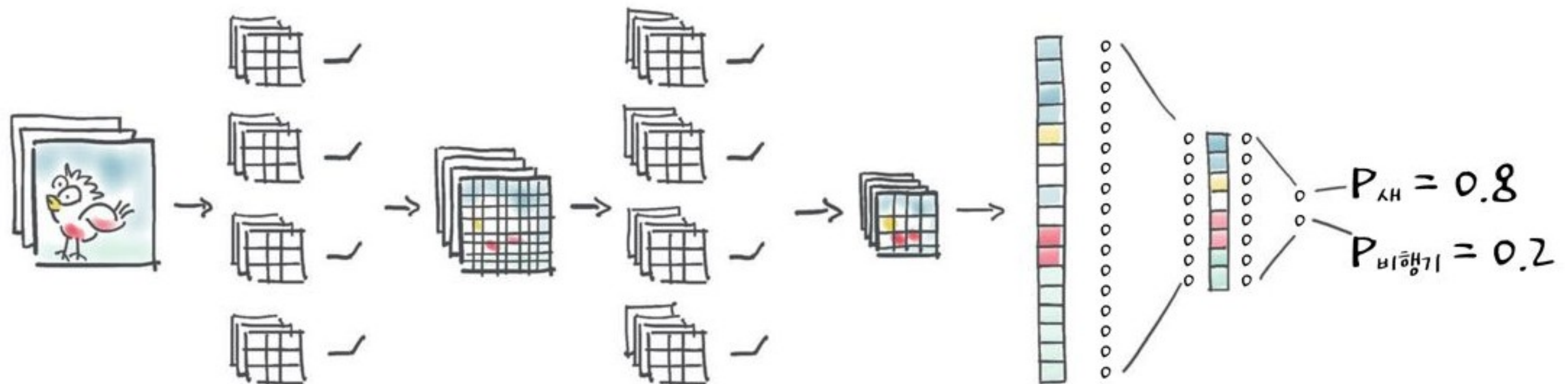


그림 8.9 우리가 지금 만들고 있는 전형적인 컨볼루션 신경망의 모양. 이미지가 여러 개의 연속적인 컨볼루션과 맥스 풀링 모듈을 거친 후 1차원 벡터로 나열되고 이내 완전 연결 모듈로 들어간다.

“뭔가 중요한 게 하나 빠졌다”는 주석은 잠시 잊자. 먼저 주목할 것은 선형 계층의 크기가 MaxPool2d의 출력 크기 $8 \times 8 \times 8 = 512$ 에 의존성을 가진다는 점이다. 이 작은 모델의 파라미터 수를 세어보자.

```
# In[24]:  
numel_list = [p.numel() for p in model.parameters()]  
sum(numel_list), numel_list
```

```
# Out[24]:  
(18090, [432, 16, 1152, 8, 16384, 32, 64, 2])
```

작은 이미지의 제한된 데이터셋을 생각하면 수긍할 만하다. 모델의 용량을 높이려면 컨볼루션층의 출력 채널 수(각 컨볼루션층이 만드는 피처의 수)를 늘려 연결되는 선형 계층도 함께 키워준다.

코드에 “경고”라는 주석을 단 이유가 있다. 모델을 실행하면 런타임 오류가 나기 때문이다.

```
# In[25]:  
model(img.unsqueeze(0))  
  
# Out[25]:  
...  
RuntimeError: size mismatch, m1:  
➡ [64 x 8], m2: [512 x 32] at c:\...\THTensorMath.cpp:940
```

오류 메시지가 다소 불분명해 보이지만 아예 무슨 얘기인지 모를 정도는 아니다. 호출 추적 정보에서 linear 레퍼런스를 찾을 수 있다. 모델을 다시 살펴보면 512×32 텐서를 가지는 모듈은 마지막 컨볼루션 블록 다음에 오는 첫째 선형 모듈인 `nn.Linear(512, 32)`뿐이다.

여기에서 하나 깜빡한 부분은 8채널의 8×8 이미지를, 512요소를 가진 1차원 벡터(배치 차원을 무시한다면 1차원이 맞다)로 차원 정보를 변경해야 하는 부분이다. 마지막 `nn.MaxPool2d`의 출력에 대해 `view`를 호출하면 해결할 수 있다. 하지만 안타깝게도 `nn.Sequential`을 사용할 때는 각 모듈의 출력을 명시적으로 볼 수 없다.⁴

8.3 nn.Module 서브클래싱하기

신경망을 개발하다 보면 기존에 만들어진 모듈에서 지원하지 않는 연산이 필요할 때가 있다. 지금의 차원 정보 변경도 비슷하지만⁵ 8.5.3절에서도 잔차^{residual} 연결을 구현하기 위해서도 직접 만들 수 있어야 한다. 그래서 이 절에서는 nn.Module의 서브클래스를 직접 만드는 법을 배워서 이미 만들어져 있는 것이나 nn.Sequential처럼 사용할 수 있게 할 예정이다.

계층 뒤에 다른 계층을 붙이는 것보다 더 복잡한 일을 수행하는 모델을 만들려면 유연성을 위해 nn.Sequential을 버려야 한다. 파이토치에서는 nn.Module 서브클래싱으로 모델에서 어떤 연산이든 수행 가능하게 해준다.

`nn.Module`을 서브클래싱하려면 먼저 `forward` 함수를 정의하여 모듈로 입력을 전달하고 출력을 반환하게 해야 한다. 이 부분이 모듈의 연산을 정의하는 영역이다. `forward`를 보면 이 책의 앞부분이 생각날 것이다. 5.5.1절에서 순방향과 역방향 경로를 정의해야 했던 모듈이 있었다. 파이토치에서는 표준 `torch` 연산을 사용하기만 한다면 자동미분 기능이 자동으로 역방향 경로를 만들어준다. 그래서 `nn.Module`에는 `backward`가 필요 없다.

우리가 만들 연산도 결국은 컨볼루션 같은 이미 있는 모듈이나 커스텀 모듈을 사용한다. 이 서브모듈 `submodule`을 포함하려면 생성자 `__init__`에 정의하고 `self`에 할당해서 `forward` 함수에서 사용할 수 있게 만들어야 한다. 이렇게 하면 모듈이 동작하는 동안 파라미터가 유지될 것이다. 모든 코드에 앞서 `super().__init__()`를 호출해야 함을 잊지 말자(잊어버릴 경우에는 파이토치가 알려줄 것이다).

8.3.1 nn.Module로 정의된 우리의 신경망

우리의 신경망을 서브모듈로 작성해보자. 먼저 nn.Conv2d와 nn.Linear 등 앞서 nn.Sequential에 넘겼던 모든 것을 생성자에 넣고 이 인스턴스들을 forward에서 순서대로 사용하자.

```
# In[26]:
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.act1 = nn.Tanh()
        self.pool1 = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(16, 8, kernel_size=3, padding=1)
        self.act2 = nn.Tanh()
        self.pool2 = nn.MaxPool2d(2)
        self.fc1 = nn.Linear(8 * 8 * 8, 32)
        self.act3 = nn.Tanh()
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = self.pool1(self.act1(self.conv1(x)))
        out = self.pool2(self.act2(self.conv2(out)))
        out = out.view(-1, 8 * 8 * 8)  ← 앞에서 놓쳤던 차원 정보 변경
        out = self.act3(self.fc1(out))
        out = self.fc2(out)
        return out
```


Net 클래스는 앞에서 만들었던 `nn.Sequential` 모델과 같은 서브모듈이다. 다만 명시적으로 `forward` 함수를 작성했기 때문에 `self.pool3`의 출력을 직접 조작해서 `view`를 통해 $B \times N$ 벡터로 만들 수 있다. 배치에 얼마나 많은 샘플이 들어있을지 알 수 없기 때문에 `view` 호출시 배치 차원을 -1로 두었다.

그리고 `nn.Module`의 서브클래스는 모델 전체를 담는 데 활용했다. 더 복잡한 신경망을 위한 새로운 빌딩 블록을 정의하기 위해 이 서브클래스를 사용할 수도 있다. 6장의 다이어그램 방식을 따라, 어디에 어떤 정보를 표현할지는 임의로 정해서 그림 8.10에 그려봤다.

NET

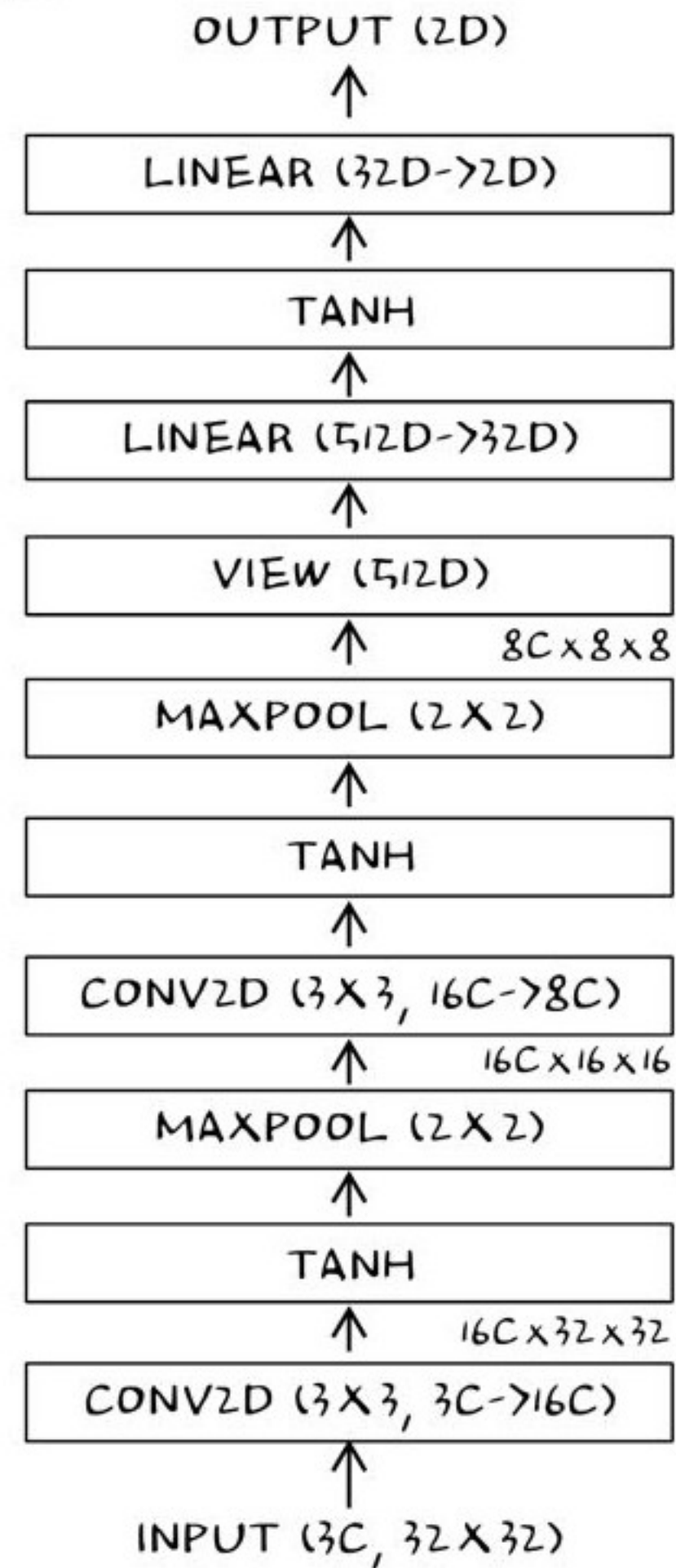


그림 8.10 우리의 기초가 되는 컨볼루션 신경망 아키텍처

분류 신경망의 목적은 일반적으로 큰 수의 픽셀을 가진 이미지에서 출발해 정보를 압축해가면서 분류 클래스(의 확률 벡터)로 만들어 가는 것이다. 모델의 아키텍처는 이 목적에 따라 두 가지 설명할 점이 있다.

첫째로, 중간에 나타나는 값의 개수가 점점 줄어드는 모습에 우리가 목표하는 바가 반영되어 있다. 컨볼루션의 채널 수가 점점 줄어들고 풀링에서 픽셀 수가 줄어들며 선형 계층에서는 입력 차원보다 낮은 수의 차원을 출력한다. 이러한 점은 분류 신경망의 전형적인 특징이다. 하지만 2장에서 살펴본 레즈넷^{ResNet} 같은 인기 있는 모델 같은 경우, 8.5.3절에서 더 다루겠지만 풀링 시 공간 분해능^{spatial resolution} 감소가 있더라도 채널의 수는 늘어난다(전체 사이즈는 결과적으로 감소한다). 빠르게 정보를 축소하는 패턴은 제한된 깊이의 신경망과 작은 이미지에 대해서는 잘 동작하지만, 신경망이 깊어질수록 감소는 천천히 일어난다.

두 번째는, 최초의 컨볼루션에서는 입력 크기에 대해 출력 크기가 줄어들지 않는다는 점이다. 만일 출력 픽셀 하나를 32개 요소(채널)를 가진 벡터로 본다면 27개의 요소($3\text{채널} \times 3 \times 3$ 커널 크기를 가지는 컨볼루션)를 가지는 선형 변환에 해당하며 어느 정도 증가하는 셈이다. 레즈넷에서 초기 컨볼루션은 147($3\text{채널} \times 7 \times 7$ 커널 크기)개의 요소로부터 64개 채널을 만들어낸다.⁶ 이 때문에 첫 계층은 예외적으로 여기를 통과하는 데이터의 전체 차원을 크게 증가시킨다(채널에 픽셀을 곱한 정도로). 하지만 출력된 픽셀을 개별적으로 생각해보면 대략 입력만큼의 출력을 가진다고 생각할 수 있다.⁷

8.3.2 파이토치가 파라미터와 서브모듈을 유지하는 방법

앞서의 코드에서는 생성자 안에서 `nn.Module`의 속성에 `nn.Module`의 인스턴스를 할당했었다. 이렇게 하면 재미있게도 모듈이 서브모듈로 등록된다.

노트

서브모듈은 tap레벨 속성이어야 하며 list나 dict 인스턴스에 들어 있으면 안 된다! 그렇지 않으면 옵티마이저가 서브모듈(과 파라미터)를 찾지 못한다. 모델에서 list나 dict를 꼭 사용해야 한다면 파이토치가 제공하는 `nn.ModuleList`나 `nn.ModuleDict`를 사용하자.

`nn.Module` 서브클래스의 메소드라면 언제나 호출 가능하다. 예를 들어 훈련 과정이 실제 사용(예측) 때와 매우 다른 경우, 별도로 `predict` 메소드를 만드는 게 나을 수도 있다. 이런 메소드의 호출은 모듈 자체보다는 `forward` 메소드를 호출하는 것과 비슷해서, `hook` 호출을 놓치게 되고 JIT는 사용시 모듈 구조를 볼 수 없다. `__call__`을 호출하지 않기 때문인데, 6.2.1절에서 다룬 것과 같은 맥락이다.

이렇게 하면 Net은 사용자의 추가 행위 없이 서브모듈의 파라미터에 접근할 수 있게 된다.

```
# In[27]:
```

```
model = Net()
```

```
numel_list = [p.numel() for p in model.parameters()]
```

```
sum(numel_list), numel_list
```

```
# Out[27]:
```

```
(18090, [432, 16, 1152, 8, 16384, 32, 64, 2])
```


코드의 `parameters()` 호출은 생성자에서 속성으로 할당된 모든 서브모듈을 찾아 이들의 `parameters()`를 재귀적으로 호출한다. 서브모듈이 얼마나 중첩되어 있든지 간에 어떤 `nn.Module`도 모든 자식 파라미터의 리스트에 접근할 수 있다. 옵티마이저는 `autograd`로 얻을 수 있는 이들의 `grad` 속성에 접근해서 손실값을 줄이기 위해 파라미터를 어떻게 바꿔야 하는지 알 수 있다. 이 내용은 앞서 5장에서 다뤘다.

이제 모듈을 직접 만드는 법을 알았으며, 이런 기법은 2부에서 많이 필요할 것이다. `Net` 클래스 구현을 되돌아보고 생성자에서 서브모듈을 등록하는 과정을 생각해보면 파라미터가 없는 `nn.Tanh`나 `nn.MaxPool2d` 같은 서브 모듈은 굳이 등록할 필요가 없을 것 같다. 이들은 `view` 호출처럼 그냥 `forward` 함수에서 직접 호출하는 게 더 쉽지 않을까?

8.3.3 함수형 API

당연히 그렇다! 그런 이유로 파이토치는 모든 nn 모듈에 대한 함수형^{functional}을 제공한다. 여기에서 ‘함수형’이란 내부 상태가 없다는 의미다. 즉 출력값이 전적으로 입력 인자에 의존한다. 실제로 `torch.nn.functional`은 nn에서 찾을 수 있는 모듈과 동일한 함수를 많이 제공한다. 다만 모듈에 저장된 파라미터와 입력 인자로 동작하는 것과는 달리 함수형이므로, 호출시 인자로 입력값 외에 파라미터도 받는다. 예를 들어 `nn.Linear`의 함수형은 `nn.functional.linear`인데, 함수 시그니처가 `linear(input, weight, bias=None)`이다. 함수 인자로 `weight`과 `bias`가 넘어간다.

다시 모델로 돌아가서 생각해 보면 nn 모듈의 `nn.Linear`와 `nn.Conv2d`를 사용해서 Net이 훈련하는 동안 Parameter를 관리하는 것은 어찌 보면 당연하다. 하지만 풀링과 활성화 함수의 경우 파라미터가 없으므로 안전하게 함수형으로 대체 가능하다.

```
# In[28]:
import torch.nn.functional as F

class Net(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 8, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(8 * 8 * 8, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.tanh(self.conv1(x)), 2)
        out = F.max_pool2d(torch.tanh(self.conv2(out)), 2)
        out = out.view(-1, 8 * 8 * 8)
        out = torch.tanh(self.fc1(out))
        out = self.fc2(out)
        return out
```

이 코드가 8.3.1절에서 정의한 Net과 동일하면서도 훨씬 더 간결하다. 생성자에서의 초기화를 위해 파라미터가 필요한 모듈의 인스턴스를 만들어 두는 부분은 여전히 필요하다.

팁

버전 1.0의 torch.nn.functional에는 tanh 같은 범용 과학 함수가 아직 존재하지만 이후에는 최상위 레벨의 torch 네임스페이스의 함수를 사용하게 되어 없어졌다. 반면 max_pool2d 같은 덜 범용적인 함수는 아직 torch.nn.functional에 남아 있다.

함수적인 방식으로 말미암아 결국 `nn.Module` API가 무엇인지도 분명해진다. `Module`은 `Parameter`라는 모습으로 상태를 저장하고 순방향 전달에 필요한 명령 집합을 서브모듈로 저장하는 컨테이너로 정의할 수 있다.

언제 함수형을 사용하고 언제 모듈 API를 활용할지는 각자 취향의 문제다. 신경망이 매우 단순하여 `nn.Sequential`을 쓰고 싶어진다면, 모듈이 답이다. 직접 순방향 흐름을 작성하는 경우라면, 파라미터로 상태를 저장할 필요가 없는 함수형 인터페이스를 사용하는 편이 낫다.

15장에서는 양자화^{quantization}를 간단히 다룬다. 활성화 같은 상태가 필요 없는 요소가 갑자기 상태가 필요한 요소로 바뀌는 이유는 양자화에 대한 정보를 포착할 필요가 있기 때문이다. 즉 모델을 양자화하되 non-JIT 스타일의 양자화라면 모듈 API를 사용하는 편이 낫다. 개인 취향이라고는 했지만 사용하다 보면 예상치 못한 동작을 보일 때가 있으니 주의해야 할 점이 있다. `nn.HardTanh`나 `nn.ReLU`처럼 상태 없는^{stateless} 모듈이 여러 애플리케이션에서 필요한 경우 각각을 별도의 인스턴스로 두는 것이 좋다. 머리를 굴리며 같은 모듈 인스턴스를 재사용하면 표준 파이토치 용법에서는 정확한 결과를 보여준다 하더라도 모델을 분석하는 도구에서 문제를 일으킨다.

이제 우리는 필요하면 직접 `nn.Module`을 만들 수도 있다. 인스턴스를 만들어 두고 `nn.Module`을 호출하는 게 부담스러운 경우를 위한 함수형 API도 알게 됐다. 이것이 파이토치로 만들어진 신경망 코드 구조의 동작을 이해하기 위한 마지막 부분이다.

이제 모델이 동작하는지 점검하고 훈련 루프로 넘어가보자.

```
# In[29]:  
model = Net()  
model(img.unsqueeze(0))  
  
# Out[29]:  
tensor([[ -0.0157,  0.1143]], grad_fn=<AddmmBackward>)
```

숫자 두 개를 얻었다! 정보가 제대로 흘러간다. 지금은 이해하지 못할 수도 있겠지만 더 복잡한 모델에서는 최초의 선형 계층의 크기를 정확하게 잡는 부분이 모든 문제를 해결하는 근원이 되기도 한다. 실제로, 유명한 어떤 연구원이 역으로 아무 숫자나 넣어놓고 에러 메시지를 유발시켜서 파이토치가 분석한 선형 계층의 크기를 파악했다는 이야기도 들은 적이 있다. 어처구니없다고? 아니, 충분히 가능한 방법이다!

8.4 우리가 만든 컨볼루션 신경망 훈련시키기

이제는 훈련 루프를 완벽하게 구성할 시점이다. 5장에서 이미 전체 구조를 개발했고 훈련 루프는 6장에서 보던 것과 매우 흡사하다. 하지만 정확도 추적을 위해 조금만 수정해보자. 모델을 실행한 다음에는 이왕이면 빠르게 학습하기 위해 GPU에서 고속으로 돌리는 법을 알려주겠다. 먼저 훈련 루프를 보자.

먼저 우리가 만든 컨볼루션 신경망은 중첩된 두 루프를 가지고 있음을 주목하자. 바깥 루프는 에포크 단위로 돌며, 안쪽 루프는 Dataset에서 배치를 만드는 DataLoader 단위로 돈다.

각 루프에서 우리는,

1. 모델에 입력값을 넣고(순방향 전달)
2. 손실값을 계산하고(마찬가지로 순방향 전달)
3. 이전 기울기값을 0으로 리셋하고
4. `loss.backward()`를 호출하여 모든 파라미터에 대한 손실값의 기울기를 계산한다(역방향 전달).
5. 이후 옵티마이저를 통해 손실값을 낮추도록 파라미터를 조정한다.

동시에 몇가지 정보를 수집해서 출력도 한다. 다음의 훈련 루프는 7장과 거의 동일하지만 다시 한번 각각이 무엇을 하는지 기억해보자.

```

# In[30]:
import datetime  ← 파이썬에 포함된 datetime 모듈을 사용함

def training_loop(n_epochs, optimizer, model, loss_fn, train_loader):
    for epoch in range(1, n_epochs + 1):  ← 에포크 숫자는 0 대신, 1부터 n_epochs까지 루프를 돌며 부여함
        loss_train = 0.0
        for imgs, labels in train_loader:  ← 데이터 로더가 만들어준 배치 안에서 데이터셋을 순회함

            outputs = model(imgs)  ← 모델에 배치를 넣어줌

            loss = loss_fn(outputs, labels)  ← 그리고 최소화하려는 손실값을 계산

            optimizer.zero_grad()  ← 마지막에 이전 기울기 값을 지움

            loss.backward()  ← 역전파 수행. 즉 신경망이 학습할 모든 파라미터에 대한 기울기를 계산함

            optimizer.step()  ← 모델 업데이트

            loss_train += loss.item()  ← 에포크 동안 확인한 손실값을 모두 더한다.
                                     ← 기울기값을 꺼내고자 .item()을 사용해 손실값을
                                     ← 파이썬 수로 변환하는 것은 중요하므로 잘 기억해두자.

        if epoch == 1 or epoch % 10 == 0:
            print('{} Epoch {}, Training loss {}'.format(
                datetime.datetime.now(), epoch,
                loss_train / len(train_loader)))  ← 배치 단위의 평균 손실값을 구하기 위해
                                                ← 훈련 데이터 로더의 길이로 나눈다.
                                                ← 이 값이 총합보다 훨씬 직관적이다.

```

7장처럼 Dataset을 사용하고 DataLoader로 래핑하여 이전처럼 신경망과 옵티마이저, 손실 함수를 인스턴스화하고 훈련 루프를 호출한다.

이제 7장과 다른 점은 모델이 커스텀 nn.Module 서브클래스를 통해 컨볼루션을 사용한다는 점이다. 손실값을 출력하면서 100에포크만큼 훈련시켜보자. 하드웨어에 따라 끝나는 데 20분이 넘게 걸릴지도 모른다!

```
# In[31]:
train_loader = torch.utils.data.DataLoader(cifar2, batch_size=64,
                                           shuffle=True)

model = Net() # ← 신경망을 초기화하고...
optimizer = optim.SGD(model.parameters(), lr=1e-2) # ← ... 앞서 다뤄본
                                                    확률적 경사 하강 옵티마이저
loss_fn = nn.CrossEntropyLoss() # ← 7.10절에서 살펴본 크로스엔트로피 손실값

training_loop( ← 앞에서 정의한 훈련 루프를 호출
    n_epochs = 100,
    optimizer = optimizer,
    model = model,
    loss_fn = loss_fn,
    train_loader = train_loader,
)
```

```
# Out[31]:
```

```
2020-01-16 23:07:21.889707 Epoch 1, Training loss 0.5634813266954605
2020-01-16 23:07:37.560610 Epoch 10, Training loss 0.3277610331109375
2020-01-16 23:07:54.966180 Epoch 20, Training loss 0.3035225479086493
2020-01-16 23:08:12.361597 Epoch 30, Training loss 0.28249378549824855
2020-01-16 23:08:29.769820 Epoch 40, Training loss 0.2611226033253275
2020-01-16 23:08:47.185401 Epoch 50, Training loss 0.24105800626574048
2020-01-16 23:09:04.644522 Epoch 60, Training loss 0.21997178820477928
2020-01-16 23:09:22.079625 Epoch 70, Training loss 0.20370126601047578
2020-01-16 23:09:39.593780 Epoch 80, Training loss 0.18939699422401987
2020-01-16 23:09:57.111441 Epoch 90, Training loss 0.17283396527266046
2020-01-16 23:10:14.632351 Epoch 100, Training loss 0.1614033816868712
```

이제 신경망을 훈련시킬 수 있다. 하지만 새를 관찰하는 친구는 우리가 매우 낮은 훈련 손실값까지 훈련을 수행했다는 말에도 별다른 감동을 받지 않을 것 같다.

8.4.1 정확도 측정

손실값보다 해석하기 좋은 측정 방법으로 훈련셋과 검증셋을 통한 정확도 측정을 생각해볼 수 있다. 7장과 동일한 코드를 사용하자.


```

# In[32]:
train_loader = torch.utils.data.DataLoader(cifar2, batch_size=64, shuffle=False)
val_loader = torch.utils.data.DataLoader(cifar2_val, batch_size=64, shuffle=False)

def validate(model, train_loader, val_loader):
    for name, loader in [("train", train_loader), ("val", val_loader)]:
        correct = 0
        total = 0

        with torch.no_grad():  ← 파라미터를 업데이트하지 않을 것이므로 기울기는 필요 없다.
            for imgs, labels in loader:
                outputs = model(imgs)
                _, predicted = torch.max(outputs, dim=1)  ← 가장 높은 값을 가진 인덱스를 출력한다.
                total += labels.shape[0]  ← 예제 수를 세어서 total을 배치 크기만큼 증가시킨다.
                correct += int((predicted == labels).sum())  ← 예측값이 실측값에 맞은 경우가 배치에서 얼마나 나왔는지 세어 합한다.

            print("Accuracy {}: {:.2f}".format(name, correct / total))

validate(model, train_loader, val_loader)

# Out[32]:
Accuracy train: 0.93
Accuracy val: 0.89

```

정수 텐서를 파이썬 int 형으로 캐스팅한다. 훈련 루프에서 했던 `.item()`을 사용한 것과 같다.

79%의 정확도를 보인 완전 연결 모델보다 성능이 훨씬 좋아졌다. 검증셋에서 오차를 거의 절반 가까이 줄였다. 동시에, 더 적은 파라미터를 사용했다. 이미지에 있는 물체를 인식하는 작업이 지역성이나 평행이동 불변성에 대응하여 더 일반화되었다. 에포크를 늘려 더 학습해보고 성능을 얼마나 더 끌어올릴 수 있는지 확인해보자.

8.4.2 모델을 저장하고 불러오기

이 정도면 만족할 만한 모델이므로, 이제 저장해보면 되지 않을까? 저장은 쉽다. 파일로 모델을 저장해보자.

```
# In[33]:  
torch.save(model.state_dict(), data_path + 'birds_vs_airplanes.pt')
```

이제 birds_vs_airplanes.pt 파일에는 model의 모든 파라미터가 들어 있다. 즉 두 개의 컨볼루션 모듈과 두 개의 선형 모델에 포함된 가중치와 편향값을 저장하고 있으며, 모델 구조는 포함되어 있지 않다. 따라서 친구를 위해 모델을 실제 서비스 목적으로 제공한다면 model 클래스도 보관해뒀다가 인스턴스를 만들고 파라미터를 파일에서 읽어와야 한다.


```
# In[34]:  
loaded_model = Net()  ← 모델 상태를 저장하고 나중에 읽는 사이에,  
                        Net 정의가 바뀌지 않았는지 반드시 확인하기 바란다.  
loaded_model.load_state_dict(torch.load(data_path + 'birds_vs_airplanes.pt'))  
  
# Out[34]:  
<All keys matched successfully>
```

코드 저장소의 ../data/p1ch7/birds_vs_airplanes.pt 폴더에 사전 훈련된 모델을 저장해 놓았으니 참고 바란다.

8.4.3 GPU에서 훈련시키기

이제 신경망도 있고 훈련도 할 수 있다! 훈련을 더 빠르게 할 수 있다면 더 좋을 것이다. 이를 위해 훈련을 GPU에서 수행하는 건 그다지 놀랄 일도 아니다. 3장에서 살펴본 `.to` 메소드를 사용하면 데이터 로더에서 얻은 텐서를 GPU로 옮길 수 있다. 이러면 연산 작업은 자연스럽게 GPU 위에서 이루어진다. 당연히 파라미터도 GPU로 옮길 필요가 있다. 다행히 `nn.Module`도 `.to` 메소드가 있어서 모든 파라미터를 GPU로 옮길 수 있다(혹은 `dtype` 인자를 전달할 때 타입을 변경할 수 있다).

`Module.to`와 `Tensor.to`는 사실 조금 다르다. `Module.to`는 모듈 인스턴스 자체를 수정한다. 하지만 `Tensor.to`는 (연산 중에서는 `Tensor.tanh`와 유사하게) 새 텐서를 반환한다. 파라미터를 원하는 디바이스로 이동 후 `Optimizer`를 만드는 식의 구현을 추천한다.

가능하다면 GPU로 옮겨 작업하는 방식도 추천한다. `torch.cuda.is_available` 값에 따라 `device` 변수를 설정하는 것이 좋은 구현 패턴이다.

```
# In[35]:  
device = (torch.device('cuda') if torch.cuda.is_available()  
          else torch.device('cpu'))  
print(f"Training on device {device}.")
```

이러면 `Tensor.to` 메소드로 데이터 로더가 올려준 텐서를 GPU로 옮기도록 훈련 루프를 수정할 수 있다. 코드는 입력을 GPU로 옮기는 두 줄을 제외하면 8.4절 초반에서 살펴본 첫 번째 버전의 코드와 완전 동일하다.


```

# In[36]:
import datetime

def training_loop(n_epochs, optimizer, model, loss_fn, train_loader):
    for epoch in range(1, n_epochs + 1):
        loss_train = 0.0

        for imgs, labels in train_loader:
            imgs = imgs.to(device=device)  ← imgs와 labels를 옮기는 이 두 줄만 이전과 다르다.
            labels = labels.to(device=device)
            outputs = model(imgs)
            loss = loss_fn(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            loss_train += loss.item()

        if epoch == 1 or epoch % 10 == 0:
            print('{} Epoch {}, Training loss {}'.format(
                datetime.datetime.now(), epoch,
                loss_train / len(train_loader)))

```

validate 함수에서도 동일한 방식의 수정이 필요하다. 이후 모델을 인스턴스화하고 device로 옮긴 후 이전처럼 실행한다.⁸

```
# In[37]:
```

```
train_loader = torch.utils.data.DataLoader(cifar2, batch_size=64,  
                                           shuffle=True)
```

```
model = Net().to(device=device) ←  
optimizer = optim.SGD(model.parameters(), lr=1e-2)  
loss_fn = nn.CrossEntropyLoss()
```

모델(파라미터)을 GPU로 옮긴다. 모델이나 입력을 GPU로 옮기지 않으면 파이토치는 GPU와 CPU 입력을 섞어 실행하는 연산을 지원하지 않으므로 '동일 디바이스에 존재하지 않음' 에러가 발생한다.

```
training_loop(  
    n_epochs = 100,  
    optimizer = optimizer,  
    model = model,  
    loss_fn = loss_fn,  
    train_loader = train_loader,  
)
```

Out[37]:

```
2020-01-16 23:10:35.563216 Epoch 1, Training loss 0.5717791349265227
2020-01-16 23:10:39.730262 Epoch 10, Training loss 0.3285350770137872
2020-01-16 23:10:45.906321 Epoch 20, Training loss 0.29493294959994637
2020-01-16 23:10:52.086905 Epoch 30, Training loss 0.26962305994550134
2020-01-16 23:10:56.551582 Epoch 40, Training loss 0.24709946277794564
2020-01-16 23:11:00.991432 Epoch 50, Training loss 0.22623272664892446
2020-01-16 23:11:05.421524 Epoch 60, Training loss 0.20996672821462534
2020-01-16 23:11:09.951312 Epoch 70, Training loss 0.1934866009719053
2020-01-16 23:11:14.499484 Epoch 80, Training loss 0.1799132404908253
2020-01-16 23:11:19.047609 Epoch 90, Training loss 0.16620008706761774
2020-01-16 23:11:23.590435 Epoch 100, Training loss 0.15667157247662544
```


우리가 다루고 있는 신경망이 작음에도 불구하고, GPU를 사용했더니 속도가 더욱 개선됐다. GPU 연산은 크기가 큰 모델에서 그 장점이 확연히 드러난다.

신경망의 가중치를 읽어올 때 문제가 하나 있다. 파이토치는 가중치를 저장할 때 어떤 디바이스를 기억해뒀다가 반대로 읽어들이는 때도 해당 디바이스를 사용하므로, GPU에 있던 가중치는 나중에 GPU로 복구될 것이다. 나중에 어떤 디바이스에서 돌릴지 모르니, 신경망을 CPU로 옮긴 후에 저장하든가 아니면 파일에서 읽어들이고 후 CPU로 옮기든가 해야 한다. 이를 간단하게 하려면 가중치를 로딩할 때 파이토치가 기억하는 디바이스 정보를 덮어쓰면 된다. `torch.load` 인자에 `map_location` 키워드를 전달한다.

```
# In[39]:
loaded_model = Net().to(device=device)
loaded_model.load_state_dict(torch.load(data_path
                                       + 'birds_vs_airplanes.pt',
                                       map_location=device))

# Out[39]"
<All keys matched successfully>
```

8.5 모델 설계

우리는 간단한 모델 구현의 표준적인 `nn.Module` 서브클래스를 사용해서 모델을 만들었다. 훈련도 잘 수행했고 모델 훈련에 어떻게 GPU를 사용하는지도 살펴보았다. 그래서 이미지를 잘 분류하는 피드포워드^{feed-forward} 컨볼루션 신경망을 만들어 훈련하는 지점까지 도달했다. 그럼 이제 뭘 해야 할까? 좀 더 복잡한 문제가 나타나면 어떻게 해야 할까? 솔직히 새와 비행기를 분류하는 데이터셋은 그다지 복잡하지 않은 경우다. 예를 들면 이미지는 매우 작고, 찾을 물체가 이미지의 중앙에 위치하면서 이미지 대부분을 차지하는 형태인 것이다.

만일 이미지넷^{ImageNet}처럼 더 크고 복잡한 이미지에서 답을 찾으려면, 여러 시각적 단서를 찾기 위해 계층적인 구조화가 필요하다. 예를 들어 작은 벽돌 모양이 리모컨인지 핸드폰인지 예측을 시도할 경우, 신경망은 스크린같이 생긴 사물을 찾으려 할 것이다.

게다가 실세계의 많은 문제가 다 이미지에 대한 것은 아니다. 테이블 데이터나 시계열이나 텍스트를 다룰 수도 있다. 신경망은 적절한 아키텍처(즉 계층이나 모듈의 연결 구조)와 적절한 손실 함수가 주어질 경우, 이런 종류의 데이터를 대상으로 문제를 풀만큼 충분히 유연하다는 특징이 있다.

파이토치는 매우 다양한 모듈과 손실 함수를 담은 채로 제공되기 때문에 단순한 피드포워드 요소부터 장단기 메모리^{LSTM, long short-term memory} 모듈과 트랜스포머 신경망(이 두 아키텍처는 시계열 데이터를 위한 아키텍처로 유명하다)까지 구현할 수 있다. 몇몇 모델은 파이토치 허브에서 받을 수 있고 torchvision 일부로 들어 있기도 하며 그 외 분야별 커뮤니티에서도 구할 수 있다.

2부에서는 고급 아키텍처를 살펴볼 예정이다. CT 스캔을 처음부터 끝까지 살펴보되 일반적인 경우로 한정해서 다룰 것이다. 신경망 아키텍처에 대한 여러 변형은 이 책의 범위를 벗어난다. 하지만 파이토치의 엄청난 표현력 덕분에 여러분은 지금까지 쌓아온 지식을 활용하여 거의 대부분의 아키텍처를 구현할 수 있을 만큼 이해하게 됐다. 이번 절의 목적은 여러분에게 개념적인 도구를 제공해서 최신 논문을 읽고 바로 파이토치로 구현하거나 때때로 저자들이 논문과 함께 제공하는 파이토치 구현을 막힘없이 술술 읽어내려갈 수 있게 해주는 것이다.

8.5.1 메모리 용량 늘리기: 너비

피드포워드 아키텍처 하나를 소개한다. 복잡한 부분을 살펴보기 전에 먼저 차원 정보 몇 가지를 살펴보자. 첫 번째는 신경망의 너비^{width} 차원으로, 이는 신경망 계층 내의 뉴런 수 혹은 컨볼루션의 채널 수에 해당하는 값이다. 파이토치에서는 모델을 쉽게 넓힐 수 있다. 첫 번째 컨볼루션의 출력 채널 수를 더 크게 하고 이어지는 계층도 여기에 맞춰 키워준다. 이렇게 하면 더 길어진 벡터를 가지게 되니 이에 따라 완전 연결 계층으로 전환되는 forward 함수에도 반영한다.

```
# In[40]:
class NetWidth(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 16, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(16 * 8 * 8, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.tanh(self.conv1(x)), 2)
        out = F.max_pool2d(torch.tanh(self.conv2(out)), 2)
        out = out.view(-1, 16 * 8 * 8)
        out = torch.tanh(self.fc1(out))
        out = self.fc2(out)
        return out
```


모델 정의에서 하드코딩을 피하려면 파라미터를 init에 전달하고 너비를 파라미터화해서 forward 함수에서 view를 호출할 때도 이를 고려한다.

```
# In[42]:
class NetWidth(nn.Module):
    def __init__(self, n_chans1=32):
        super().__init__()
        self.n_chans1 = n_chans1
        self.conv1 = nn.Conv2d(3, n_chans1, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(n_chans1, n_chans1 // 2, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(8 * 8 * n_chans1 // 2, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.tanh(self.conv1(x)), 2)
        out = F.max_pool2d(torch.tanh(self.conv2(out)), 2)
        out = out.view(-1, 8 * 8 * self.n_chans1 // 2)
        out = torch.tanh(self.fc1(out))
        out = self.fc2(out)
        return out
```

각 계층에서 채널과 피처의 수를 나타내는 숫자들은 직접적으로 모델의 파라미터 수에 영향을 미친다. 다른 부분이 다 동일한 경우 이러한 변화는 모델의 용량을 증가시킨다. 전에 했던 것처럼 모델이 가진 파라미터가 얼마나 많은지는 다음과 같이 확인할 수 있다.

```
# In[44]:  
sum(p.numel() for p in model.parameters())
```

```
# Out[44]:  
38386
```

용량이 클수록 모델이 다룰 수 있는 입력은 더 다양해진다. 하지만 동시에 과적합할 가능성도 커진다. 파라미터가 많아져서 입력에서 불필요한 부분까지 기억해버릴 수 있기 때문이다. 일전에 과적합과 씨름해봤던 것처럼, 샘플 수를 늘리거나 데이터가 충분하지 않다면 동일한 데이터를 인공적으로 수정해서 증강시켜야 한다.

과적합을 제어하기 위해 (데이터가 아닌) 모델 레벨에서 다룰 수 있는 몇가지 트릭이 있는데 이 중 대표적인 것을 한번 알아보자.

8.5.2 모델이 수렴하고 일반화하도록 돕는 방법: 정규화

모델 훈련은 중요한 두 단계를 거친다. 하나는 최적화 단계로서, 훈련셋에 대해 손실값을 줄이는 경우이고, 다른 하나는 일반화 단계로서, 모델이 훈련셋뿐 아니라 이전에 겪어보지 않은 검증셋 같은 데이터에 대해서도 동작하게 하는 것이다. 이 두 단계를 위한 수학적 도구를 모아서 **정규화** regularization라는 이름을 붙이기도 한다.

■ 파라미터 제어하기: 가중치 페널티

일반화^{generalization}를 안정적으로 수행하기 위한 첫 번째 방법으로 손실값에 정규화 항을 넣는 것이 있다. 이 정규화 항을 조작해서 모델의 가중치가 상대적으로 작게 만든다. 즉 훈련을 통해 증가할 수 있는 크기를 제한하는 것이다. 큰 가중치 값에 페널티를 부과하는 셈이다. 이러면 손실값은 다소 매끄러운 등고선 형태를 띠는데 개별 샘플에 맞춰서 얻는 이득이 상대적으로 줄어들게 된다.

가장 유명한 정규화 항으로 L2 정규화와 L1 정규화를 들 수 있다. L2 정규화는 모델의 모든 가중치에 대한 제곱합이며, L1 정규화는 모델의 모든 가중치 절댓값의 합이다.⁹ 둘 다 어떤 (작은) 값으로 범위를 조정하는데 이 값은 훈련 전에 설정하는 하이퍼파라미터 중 하나다.

L2 정규화는 가중치 감쇠(weight decay)라고도 한다. 이런 이름을 가진 이유는 SGD와 역전파를 생각해보면 되는데, 파라미터 w_i 에 대한 L2 정규화의 음의 기울기는 $-2 * \lambda * w_i$ 로, 여기서 λ 는 앞에서 언급한 하이퍼파라미터에 해당하는데 파이토치에서는 간단히 가중치 감쇠라고 부른다. 그래서 손실 함수에 L2 정규화를 더하는 것은 최적화 단계에서 현재 값에 비례해서 각 가중치를 줄이는 역할을 한다(가중치 감쇠라 부르는 이유다). 가중치 감쇠는 편향값 같은 신경망의 모든 파라미터에 적용된다.

파이토치에서 정규화는 손실값에 항을 하나 추가하는 것으로 매우 손쉽게 구현할 수 있다. 손실값 계산 후 손실 함수가 어떤 것이든지 간에 모델의 각 파라미터를 돌면서 (L2를 위한) 제곱이나 (L1을 위한) 절댓값을 더하고 역전파할 수 있다.


```

# In[45]:
def training_loop_l2reg(n_epochs, optimizer, model, loss_fn, train_loader):
    for epoch in range(1, n_epochs + 1):
        loss_train = 0.0
        for imgs, labels in train_loader:
            imgs = imgs.to(device=device)
            labels = labels.to(device=device)
            outputs = model(imgs)
            loss = loss_fn(outputs, labels)

            l2_lambda = 0.001
            l2_norm = sum(p.pow(2.0).sum()
                           for p in model.parameters())
            loss = loss + l2_lambda * l2_norm

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        loss_train += loss.item()

    if epoch == 1 or epoch % 10 == 0:
        print('{} Epoch {}, Training loss {}'.format(
            datetime.datetime.now(), epoch,
            loss_train / len(train_loader)))

```

← L1 정규화라면 pow(2.0)을 abs()로 바꾼다.

그런데 파이토치의 SGD 옵티마이저에는 이미 `weight_decay` 파라미터가 있고 $2 * \text{lambda}$ 값을 가지며 업데이트 동작시 앞에서 설명한 것처럼 가중치 감쇠를 수행한다. 손실값에 가중치의 L2 정규화를 더한 것과 동일하므로, 자동미분과 손실값에서 항을 누적하는 작업을 할 필요가 없다.

■ 입력 하나에 너무 의존하지 않기: 드랍아웃

과적합을 극복하는 효과적인 방법은 2014년 토론토의 제프리 힌튼^{Geoffrey Hinton} 그룹의 일원이었던 니티쉬 스리바스타바^{Nitish Srivastava} 등이 공저한 「드랍아웃: 신경망이 과적합을 방지하는 간단한 방법」(<http://mng.bz/nPMa>)이라는 논문에서 처음 제안되었다.¹⁰ 우리가 정확히 잘 찾아낸 듯하다. 드랍아웃^{dropout}에 대한 개념은 매우 단순하다. 훈련을 반복할 때마다 신경망의 뉴런 출력을 랜덤하게 0으로 만드는 작업을 일어나게 하는 것이다.

이 방법을 활용하면, 매 훈련 때마다 조금씩 다른 뉴런의 토폴로지가 만들어지기 때문에 신경망이 각 입력 샘플을 암기하려는 기회를 줄이므로 과적합을 방지한다. 다른 관점에서 드롭아웃은 모델이 피처를 만들려는 것을 교란해서 데이터 증강과 비슷한 효과를 내지만, 증강과 다르게 신경망 전체에 이러한 효과를 낸다.

파이토치에서는 비선형 활성화 함수와 선형 혹은 여러 계층의 컨볼루션 모듈 사이에 `nn.Dropout` 모듈을 넣어 모델의 드롭아웃을 구현할 수 있다. 인자로써, 어떤 입력이 0이 될지에 대한 확률을 지정해준다. 컨볼루션은 전용 값인 `nn.Dropout2d`나 `nn.Dropout3d`를 사용해서 해당 입력값의 채널 전부를 0으로 만든다.

```

# In[47]:
class NetDropout(nn.Module):
    def __init__(self, n_chans1=32):
        super().__init__()
        self.n_chans1 = n_chans1
        self.conv1 = nn.Conv2d(3, n_chans1, kernel_size=3, padding=1)
        self.conv1_dropout = nn.Dropout2d(p=0.4)
        self.conv2 = nn.Conv2d(n_chans1, n_chans1 // 2, kernel_size=3, padding=1)
        self.conv2_dropout = nn.Dropout2d(p=0.4)
        self.fc1 = nn.Linear(8 * 8 * n_chans1 // 2, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.tanh(self.conv1(x)), 2)
        out = self.conv1_dropout(out)
        out = F.max_pool2d(torch.tanh(self.conv2(out)), 2)
        out = self.conv2_dropout(out)
        out = out.view(-1, 8 * 8 * self.n_chans1 // 2)
        out = torch.tanh(self.fc1(out))
        out = self.fc2(out)
        return out

```


드롭아웃은 훈련 중에 활성화되고 훈련이 끝난 모델을 제품으로 사용할 때는 그냥 통과하게 두거나 확률값에 0을 넣어준다. Dropout 모듈의 train 프로퍼티를 통해 제어하면 된다. 파이토치는 nn.Model 서브클래스에 대해 model.train() 혹은 model.eval() 호출로 전환 가능함을 기억하자.

이 호출은 서브 모듈에도 자동으로 적용되므로 그중에 Dropout이 있다면 순방향 혹은 역방향 전달시 원하는 대로 모드가 전환되어 동작할 것이다.

■ 활성화 함수 억제하기: 배치 정규화

드롭아웃이 대유행하던 2015년, 구글의 세르게이 이오페Sergey Ioffe와 크리스티안 세게디Christian Szegedy는 「배치 정규화: 내부 공변량 이동을 줄여 딥러닝 훈련을 가속하기」(<https://arxiv.org/abs/1502.03167>)라는 논문을 발표했다. 이 논문에서는 훈련 시 여러 가지로 도움이 되는 기법을 설명한다. 이 논문에서는 학습률을 높이면서, 훈련은 초기화에 영향을 덜 받고 정규화처럼 동작해서, 결과적으로 드롭아웃에 대한 대체 방법을 설명한다.

배치 정규화batch normalization의 핵심은 입력 범위를 신경망의 활성화 함수로 바꿔서 미니 배치가 원하는 분포를 가지게 하는 것이다. 학습의 원리와 비선형 활성화 함수의 역할을 기억해보자. 비선형 활성화 함수를 활용하면, 함수의 임계 영역에서 입력이 활성화 함수에 너무 많이 작용하여 기울기가 소실되고 훈련이 느려지는 상황을 피할 수 있다.

실제로 배치 정규화는 미니 배치 샘플을 통해 중간 위치에서 얻은 평균과 표준편차를 사용하여 중간 입력값을 이동하고 범위를 바꾼다. 따라서 모델이 보는 개별 샘플이나 이로 인한 이후의 활성화 단계에서는 랜덤하게 뽑아 만들어진 미니 배치에서 통계에 의존한 값의 이동과 범위 조정이 반영되어 있는 상태다. 이 자체로 원칙적인 데이터 증강인 셈이다. 이 논문에서 저자진은 배치 정규화가 드랍아웃할 필요를 없애거나 줄여준다고 주장한다.

파이토치의 배치 정규화는 `nn.BatchNorm1D`, `nn.BatchNorm2d`, `BatchNorm3d` 모듈로 지원하며 각 입력 차원에 따라 골라 사용한다. 배치 정규화의 목적은 활성 함수의 입력 범위를 조정하는 것이므로 자연스럽게 선형 변환(우리의 경우에는 컨볼루션) 뒤에 위치하게 된다.


```

# In[49]:
class NetBatchNorm(nn.Module):
    def __init__(self, n_chans1=32):
        super().__init__()
        self.n_chans1 = n_chans1
        self.conv1 = nn.Conv2d(3, n_chans1, kernel_size=3, padding=1)
        self.conv1_batchnorm = nn.BatchNorm2d(num_features=n_chans1)
        self.conv2 = nn.Conv2d(n_chans1, n_chans1 // 2, kernel_size=3, padding=1)
        self.conv2_batchnorm = nn.BatchNorm2d(num_features=n_chans1 // 2)
        self.fc1 = nn.Linear(8 * 8 * n_chans1 // 2, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = self.conv1_batchnorm(self.conv1(x))
        out = F.max_pool2d(torch.tanh(out), 2)
        out = self.conv2_batchnorm(self.conv2(out))
        out = F.max_pool2d(torch.tanh(out), 2)
        out = out.view(-1, 8 * 8 * self.n_chans1 // 2)
        out = torch.tanh(self.fc1(out))
        out = self.fc2(out)
        return out

```

드롭아웃처럼 배치 정규화도 훈련 때와 추론 때는 각기 다르게 동작해야 한다. 추론 시에 출력은 모델이 이미 봤던 다른 입력의 통계에 의존하는 특정 입력을 위한 것이 되어서는 안 된다. 정규화는 계속 필요하겠지만 지금은 정규화 파라미터를 고정시켜놓자.

미니 배치가 실행되면 현재의 미니 배치에 대한 평균과 표준편차를 구하는 것과 더불어 파이토치가 전체 데이터셋에 대한 평균과 표준편차도 대략적으로 업데이트한다. 그래서 사용자는 `model.eval()`을 명시하고, 모델이 배치 정규화 모듈을 가지는 경우 추정값을 고정하고 정규화에 사용하기만 한다. 동작 중인 추정을 해제하고 다시 미니 배치 통계를 추정해가길 원한다면, 드롭아웃에서와 마찬가지로 `model.train()`을 호출한다.

8.5.3 더 복잡한 구조를 배우기 위해 깊이 파헤치기: 깊이

앞에서 모델의 용량을 크게 만들기 위해 바꿔야 할 첫 번째 차원인 너비^{width}에 대해 얘기했다. 두 번째 차원은 깊이^{depth}다. 이 책은 ‘딥’러닝에 대한 책이므로 당연히 깊이를 다뤄야 하기도 한다. 모델이 얇은 경우보다 깊은 경우가 항상 좋은 걸까? 상황에 따라 다를 테다. 모델이 깊어질수록 신경망은 더 복잡한 함수에 근사할 수 있다. 컴퓨터 비전에서 얇은 신경망은 사진에서 사람의 모습을 식별할 수 있지만 신경망이 깊어질수록 상반신의 얼굴과 얼굴 안에 있는 입까지 구별해낼 수 있다. 어떤 입력에 대해 뭔가를 말할 수 있도록 맥락을 이해하려면 모델을 깊게 만들어 계층적인 정보를 다룰 수 있게 해야 한다.

다른 관점으로 ‘깊이를 더한다’는 것은 신경망이 입력을 처리할 때 수행할 연산을 더 길게 나열할 수 있다는 의미다. 심층 신경망이 작업 실행 시 연속적인 연산을 수행하는 것으로 바라보는 관점은 마치 “사람 형체의 경계를 찾고 경계의 맨 위쪽에서 머리를 찾은 후 머리 안에서 입을 찾아라”라는 말처럼, 연속된 연산으로 만들어진 알고리즘을 생각하는 데 익숙한 소프트웨어 개발자에게 매력적일 수 있다.

■ 스킵 커넥션

모델의 깊이는 때때로 새로운 도전을 수반한다. 2015년 후반까지만 하더라도 딥러닝 모델의 계층은 20개 이상 늘어나기 어려웠다. 모델에 깊이가 더해질수록 훈련은 수렴하기 어려워진다. 역전파를 매우 깊은 심층 신경망으로 수행한다고 생각해보자. 파라미터에 대한 손실 함수의 미분은 특히 더 이전 계층의 경우 손실값과 파라미터 사이의 미분 연산의 체인 연결에서 오는 많은 수로 곱해져야 한다. 곱하는 수가 작을 수도 있고 그래서 정말 작은 값이 만들어지거나 혹은 반대로 큰 값이 만들어지거나 부동소수점 근사 과정에서 작은 값들이 사라져버릴 수도 있다. 핵심은 곱셈이 체인이 길게 이어지는 경우 기울기값에 기여하는 파라미터가 사라져버려서^{vanishing} 파라미터 같은 값들이 적절하게 업데이트되지 않기 때문에 효과 없는 훈련을 초래할 수 있다는 점이다.

2015년 12월, 카이밍 히(Kaiming He)와 공저자들은 단순한 트릭으로 매우 깊은 신경망이어도 성공적으로 훈련되는 잔차 신경망(residual network)인 레즈넷(ResNet)을 만들었다(<https://arxiv.org/abs/1512.03385>). 이로써 신경망이 수십 개의 계층에서 100개의 계층으로 늘어나는 문이 열리고, 컴퓨터 비전 벤치마크 문제의 최신 성능을 뛰어넘는 결과도 가져왔다. 우리는 2장에서 사전 훈련된 모델을 소개할 때 이 잔차 신경망을 만나봤다. 기법은 단순하다. 그림 8.11과 같이 계층의 짧은 회로 블록에 스킵 커넥션(skip connection)을 사용하는 것이다.

NETDEPTH / NETRES

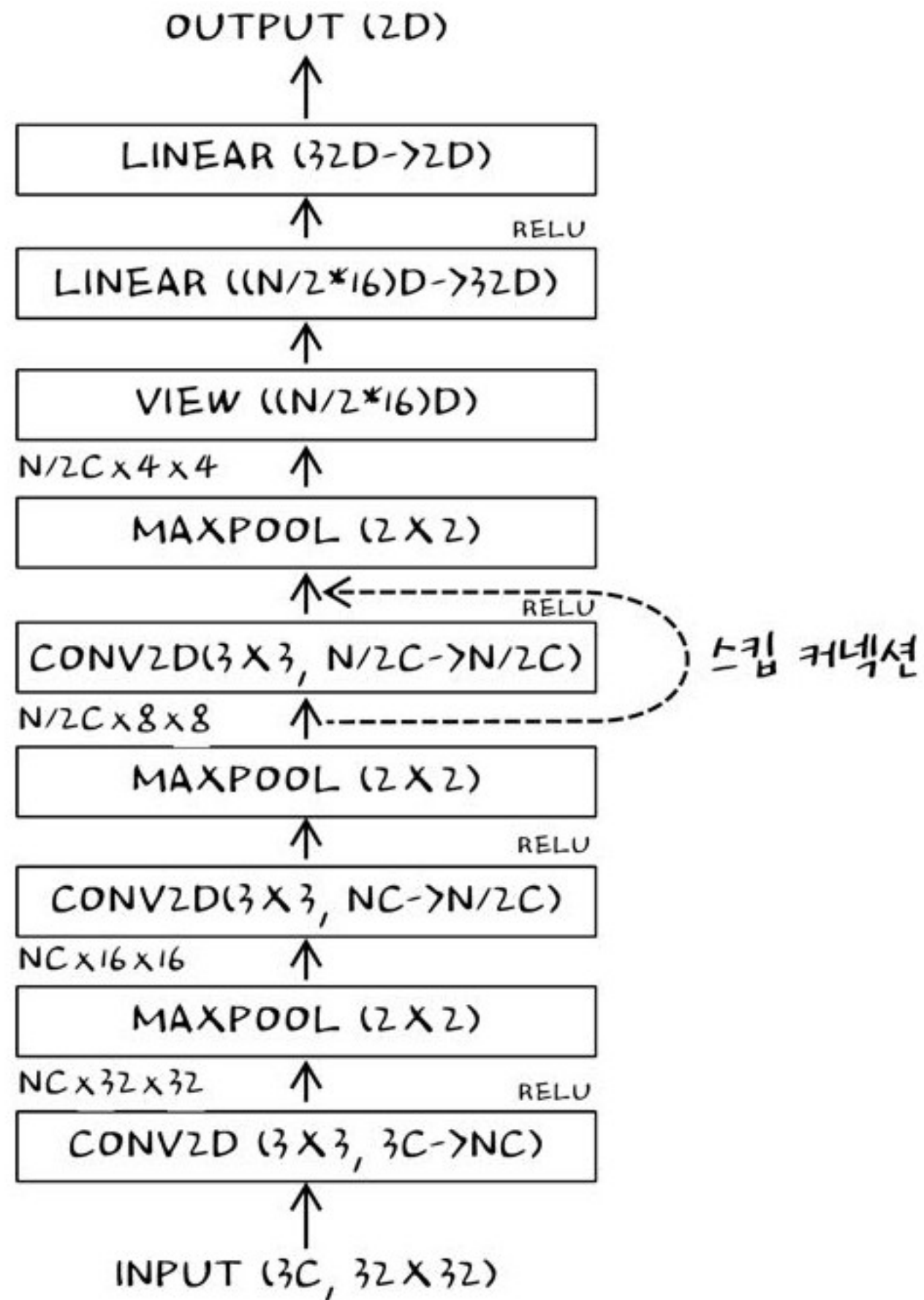


그림 8.11 세 개의 컨볼루션층을 가진 우리의 신경망 아키텍처. 여기에서 스킵 커넥션으로 NetDepth와 NetRes을 구분한다.

스킵 커넥션은 입력을 계층 블록의 출력에 연결하는 것으로서, 파이토치에서 수행할 수 있다. 우리의 단순한 컨볼루션 모델에 계층 하나를 추가하고 변화를 위한 활성화 함수로 ReLU를 사용하자. 계층이 추가된 기본적인 모듈은 다음과 같을 것이다.

```
# In[51]:
class NetDepth(nn.Module):
    def __init__(self, n_chans1=32):
        super().__init__()
        self.n_chans1 = n_chans1
        self.conv1 = nn.Conv2d(3, n_chans1, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(n_chans1, n_chans1 // 2, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(n_chans1 // 2, n_chans1 // 2, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(4 * 4 * n_chans1 // 2, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.relu(self.conv1(x)), 2)
        out = F.max_pool2d(torch.relu(self.conv2(out)), 2)
        out = F.max_pool2d(torch.relu(self.conv3(out)), 2)
        out = out.view(-1, 4 * 4 * self.n_chans1 // 2)
        out = torch.relu(self.fc1(out))
        out = self.fc2(out)
        return out
```

레즈넷처럼 스킵 커넥션을 이 모델에 추가하는 것은 forward 함수의 첫 번째 계층의 출력을 세 번째 계층의 입력에 추가하는 것과 같다.

```
# In[53]:
```

```
class NetRes(nn.Module):
    def __init__(self, n_chans1=32):
        super().__init__()
        self.n_chans1 = n_chans1
        self.conv1 = nn.Conv2d(3, n_chans1, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(n_chans1, n_chans1 // 2, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(n_chans1 // 2, n_chans1 // 2, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(4 * 4 * n_chans1 // 2, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.relu(self.conv1(x)), 2)
        out = F.max_pool2d(torch.relu(self.conv2(out)), 2)
        out1 = out
        out = F.max_pool2d(torch.relu(self.conv3(out)) + out1, 2)
        out = out.view(-1, 4 * 4 * self.n_chans1 // 2)
        out = torch.relu(self.fc1(out))
        out = self.fc2(out)
        return out
```

바꿔 말하면 표준 피드포워드 경로에 추가적으로 첫 번째 활성화 함수의 출력을 마지막 부분의 입력으로 사용하는 것이다. 이를 **아이덴티티 매핑** identity mapping이라고도 부른다. 이 방법이 어떻게 앞에서 설명한 경사 소실 vanishing gradient 문제를 완화시킬 수 있을까?

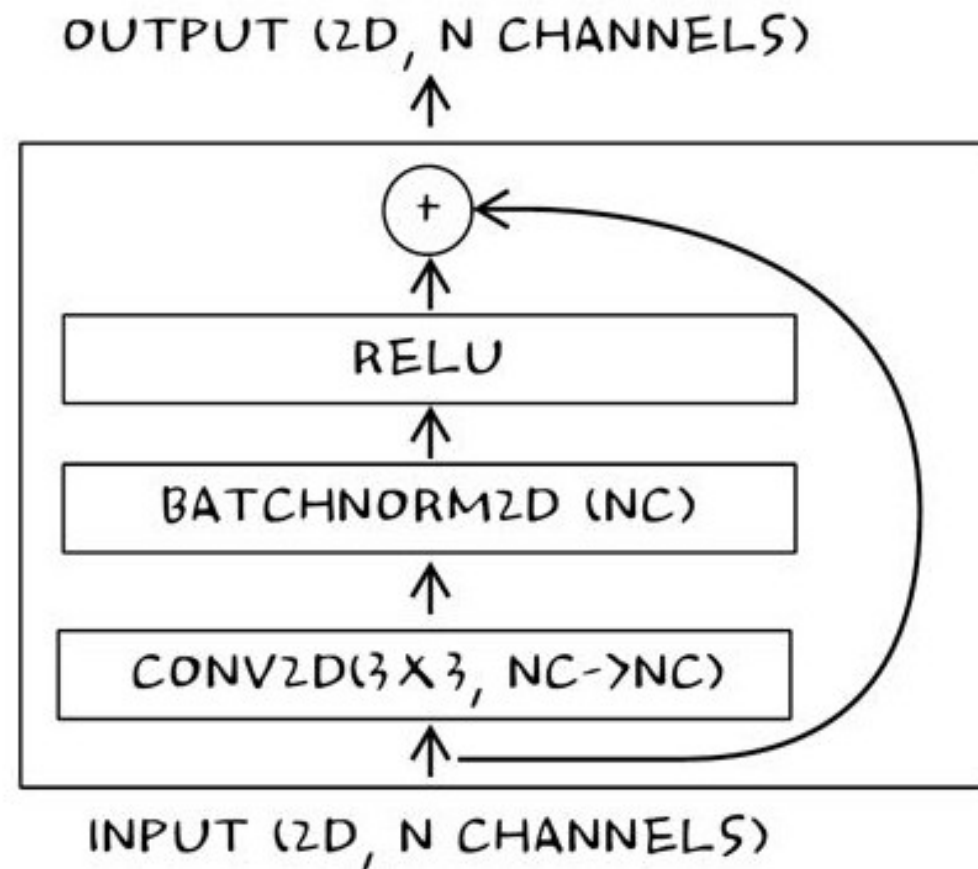
역전파를 생각해보면 스킵 커넥션 혹은 심층 신경망에서의 연속적인 스킵 커넥션은 깊은 쪽에 있는 파라미터를 손실값에 연결하는 역할을 한다. 이로 인해 체인으로 길게 연결된 여러 다른 연산들로 곱해질 기회가 줄어들고, 파라미터에 대한 손실값의 편미분으로 손실값에 대한 기울기에 더욱 직접적으로 관여한다.

사람들은 스킵 커넥션이 특히 훈련의 초반 단계에서 수렴에 도움이 되는 방향으로 작용하는 것을 발견했다. 또 잔차 신경망의 손실값 분포는 동일한 깊이나 너비를 가진 피드포워드 신경망보다 훨씬 부드럽다.

레즈넷이 등장했을 당시 스킵 커넥션이 새로운 것은 아니었다는 사실은 전혀 중요하지 않다. 하이웨이 네트워크^{Highway network}나 유넷^{U-Net} 역시 어떤 형태로든 스킵 커넥션을 사용하고 있었다. 하지만 레즈넷이 스킵 커넥션을 사용한 방식을 통해 100개 층이 넘는 깊이의 모델도 수정해서 훈련시킬 수 있는 문을 열었다.

레즈넷의 출현으로 다른 아키텍처들도 스킵 커넥션을 사용하여 다음 레벨로 진화했다. 특별히 텐스넷^{DenseNet}은 스킵 커넥션을 사용하여 각 계층을 하단의 여러 계층과 연결해서 더 적은 파라미터로 놀라운 결과를 보여주었다. 이제는 우리도 중간 단계의 출력을 하위의 다른 단계의 출력에 수학적으로 연결하여 텐스넷과 같은 모델을 만들 수 있다.

RESBLOCK



NETRESDEEP

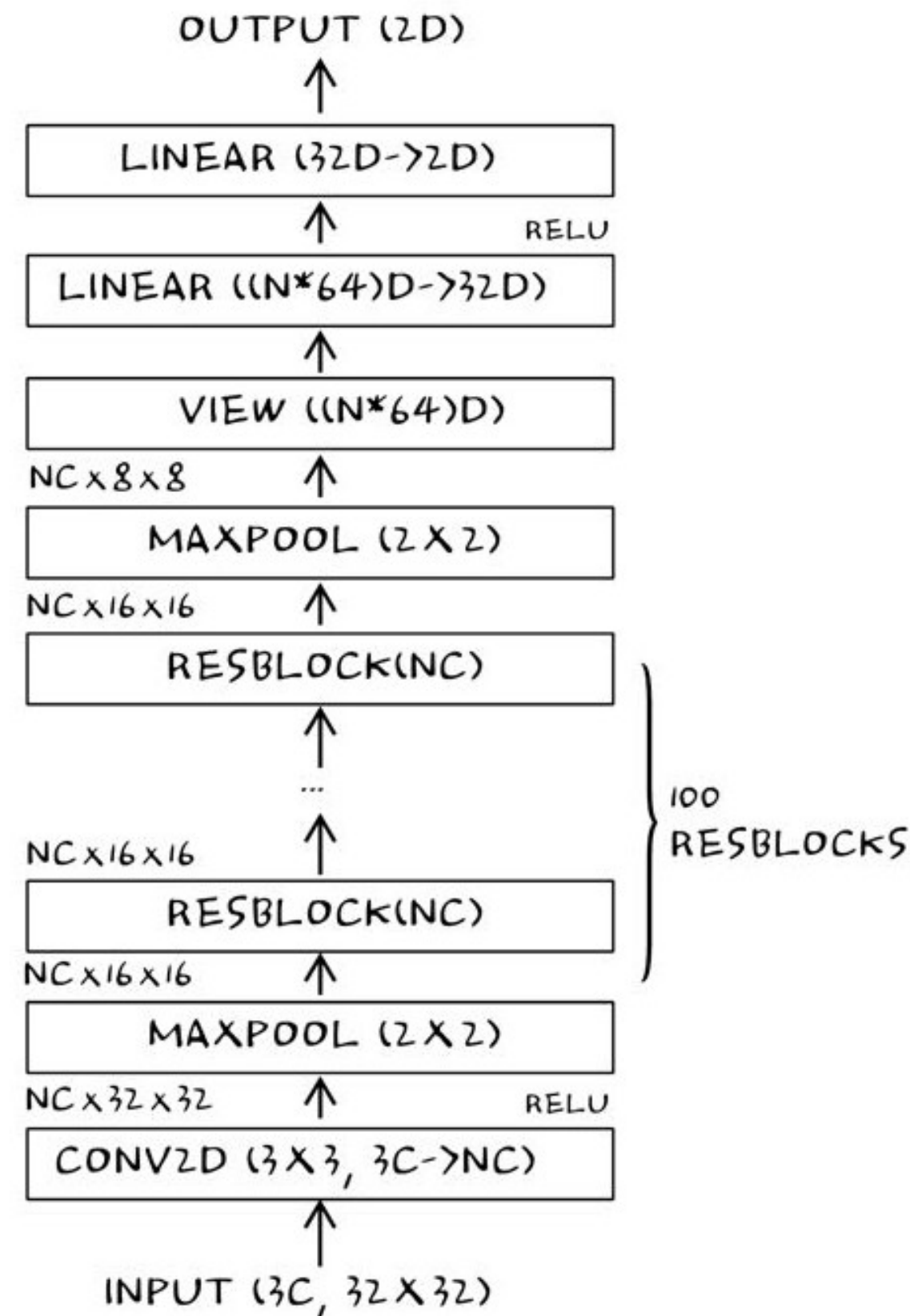


그림 8.12 잔차 커넥션을 사용한 깊은 아키텍처. 왼쪽에 간단한 잔차 블록을 정의했고 오른쪽의 신경망에서 빌딩 블록처럼 사용했다.

■ 파이토치로 매우 깊은 모델 만들기

컨볼루션 신경망에서 100개 이상의 계층을 담는 법을 설명했다. 처리 과정에서 정신을 잃지 않고 이를 파이토치에서 만들려면 어떻게 해야할까? 일반적으로 (Conv2d, ReLU, Conv2d) + skip connection 같은 빌딩 블록을 정의한 후 for 루프를 사용하여 신경망을 동적으로 만든다. 직접 살펴보자. 그림 8.12에서 묘사한 신경망을 만들어보자.

먼저 컨볼루션과 활성화 함수, 스킵 커넥션으로 이뤄진 블록을 위한 연산을 제공하는 모듈 서브클래스를 정의한다.

```
# In[55]:
```

```
class ResBlock(nn.Module):
```

```
    def __init__(self, n_chans):
```

```
        super(ResBlock, self).__init__()
```

```
        self.conv = nn.Conv2d(n_chans, n_chans, kernel_size=3,  
                               padding=1, bias=False)
```

BatchNorm 계층은
편향값의 효과를 상쇄하므로,
관례상 이를 생략한다.

```
        self.batch_norm = nn.BatchNorm2d(num_features=n_chans)
```

```
        torch.nn.init.kaiming_normal_(self.conv.weight,  
                                       nonlinearity='relu')
```

```
        torch.nn.init.constant_(self.batch_norm.weight, 0.5)
```

```
        torch.nn.init.zeros_(self.batch_norm.bias)
```

커스텀 초기화.
.kaiming_normal_은
레즈넷 논문에서처럼 표준편차를
가지는 표준 랜덤 요소로
초기화해준다. 배치 정규화는
기본값으로 평균 0과 분산 0.5를
가지는 분포의 출력을 만들도록
초기화한다.

```
    def forward(self, x):
```

```
        out = self.conv(x)
```

```
        out = self.batch_norm(out)
```

```
        out = torch.relu(out)
```

```
        return out + x
```

깊은 모델을 만들기로 했으므로 블록에 배치 정규화를 넣어 훈련 도중에 기울기 값이 없어지는 것을 방지한다. 이제 이 블록을 100개 가진 신경망을 만들어보자. 직접 복사하기와 붙여넣기를 반복해야 할까? 우리는 이미 어떻게 해야 할지를 충분히 이해하고 있다.

먼저 `init`에서 `ResBlock` 인스턴스 리스트를 포함한 `nn.Sequential`을 만든다. `nn.Sequential`을 쓰면, 한 블록의 출력을 다음 블록의 입력으로 사용할 수 있다. 또한 블록 내의 모든 파라미터를 `Net`이 볼 수 있게 해준다. 그리고 이렇게 만든 `sequential`을 `forward`에서 호출해 100개의 블록을 거쳐 출력을 만든다.

```

# In[56]:
class NetResDeep(nn.Module):
    def __init__(self, n_chans1=32, n_blocks=10):
        super().__init__()
        self.n_chans1 = n_chans1
        self.conv1 = nn.Conv2d(3, n_chans1, kernel_size=3, padding=1)
        self.resblocks = nn.Sequential(
            *(n_blocks * [ResBlock(n_chans=n_chans1)]))
        self.fc1 = nn.Linear(8 * 8 * n_chans1, 32)
        self.fc2 = nn.Linear(32, 2)

    def forward(self, x):
        out = F.max_pool2d(torch.relu(self.conv1(x)), 2)
        out = self.resblocks(out)
        out = F.max_pool2d(out, 2)
        out = out.view(-1, 8 * 8 * self.n_chans1)
        out = torch.relu(self.fc1(out))
        out = self.fc2(out)
        return out

```


실험과 재사용성을 고려하여 계층의 수를 파라미터로 지정할 수 있도록 만들었다. 두말할 필요 없이, 역전파도 예상대로 잘 동작할 것이다. 그리고 당연하게도 신경망은 수렴하는 데 꽤 걸릴 테고 수렴이 잘 안될 수도 있다. 그래서 초기화를 더 꼼꼼하게 진행한 후 NetRes를 훈련시키되 학습률도 다른 신경망에서 사용하던 $1e - 2$ 대신 $3e - 3$ 을 썼다. 신경망이 수렴할지를 고민하며 훈련시킨 적은 없었지만, 이런 식의 수정이 없다면 더 진행하기 어려울 수도 있다.

앞서 설명한 내용이 32×32 이미지 데이터셋을 위한 모델에서 깊이를 추구하라고 부추기는 것은 아니다. 다만 이미지넷같이 더욱 도전적인 데이터셋에 대해 이런 식의 접근을 어떻게 진행해야 할지는 충분히 설명했다. 동시에 torchvision의 레즈넷 같은 모델의 기존 구현을 이해하기 위해 핵심 요소가 어떤 것인지도 알게 되었다.

■ 초기화

이른 초기화에 대해 잠깐 짚고 넘어가자. 초기화는 신경망 훈련의 중요한 방법 중 하나다. 불행하게도, 나름의 이유로 파이토치의 기본 가중치 초기화는 완벽하지 않다. 그래서 여러 사람들이 개선 중이며 진전된 내용을 깃허브에서 찾아볼 수 있다(<https://github.com/pytorch/pytorch/issues/18182>). 완전히 개선되기 전까지는 가중치 초기화를 직접 고쳐야 한다. 우리 모델은 수렴하지 않았고 일반적인 초기화 방법(가중치 변화량 감소와 배치 정규화를 위한 평균값 0과 단위 분산 출력 등)도 살펴봤다. 그리고 신경망이 수렴하지 않는 경우 배치 정규화에서 출력 분산을 절반으로 줄이기도 했다.

가중치 초기화만으로도 책의 한 단원을 채울 만큼의 주제이지만 그럴 필요는 없을 것 같다. 11장에서 초기화를 다시 살펴볼 텐데, 파이토치의 기본 초기화에 대해서는 구체적으로 다루지 않으려 한다. (아마도 이 책을 다 읽기 전에) 가중치 초기화에 대한 구체적인 내용이 궁금할 정도로 많이 알게 된다면 각자 이 주제를 별도로 찾아봐도 좋을 것이다.¹¹

8.5.4 모델의 성능 비교

설계 변경의 영향을 각각 분리해서 그림 8.13에 정리했다. 특정 숫자에 과도한 의미를 부여할 필요는 없다. 문제의 설정과 실험은 단순하고 서로 다른 랜덤 시드^{seed}로 실험을 반복하면 아마 검증셋의 정확도 차이만큼 큰 변동을 일으킬지도 모른다. 설명을 위해 학습률이나 훈련에서의 에포크 수 등은 모두 동일하게 맞췄다. 실제로는 좋은 결과를 만들기 위해 이 값들을 바꿔갈 뿐 아니라 부가적인 설계 요소도 추가해야 한다.

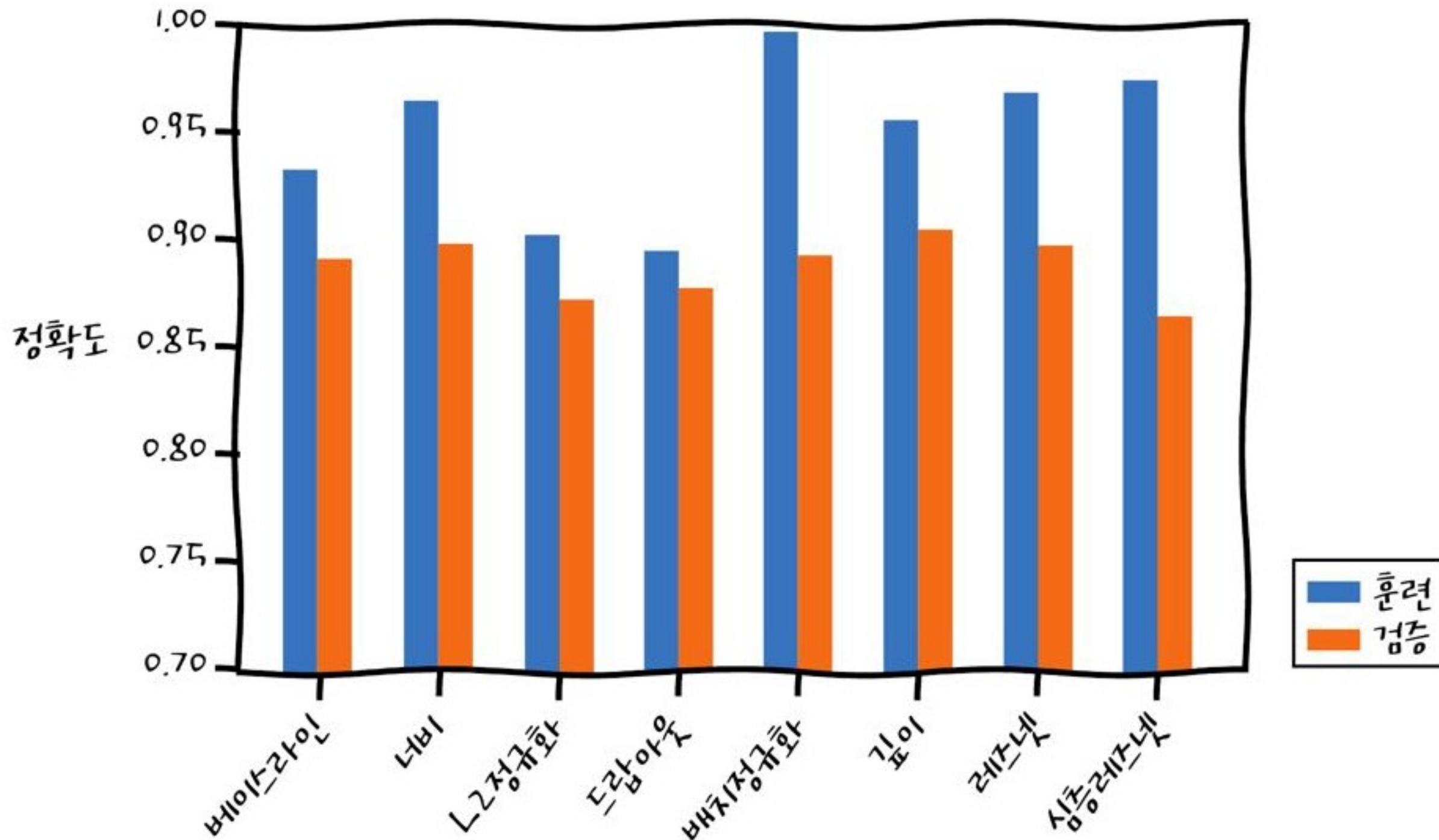


그림 8.13 수정된 신경망은 대부분 비슷한 성능을 가진다.

품질적인 측면으로는 5.5.3절에서 검증과 과적합을 다룰 때처럼 순서가 있다고 볼 수도 있다. 정규화 관점에서 배치 정규화보다 훨씬 엄격한 통계적 평가를 수행하는 가중치 감쇠와 드롭아웃 정규화는 두 정확도 사이의 차이가 훨씬 작은 것을 볼 수 있다. 배치 정규화는 모델의 수렴을 돕는 역할로, 신경망은 거의 100%의 훈련 정확도를 가진다. 그래서 반대로 가중치 감쇠와 드롭아웃은 정규화로 해석한다.

8.5.5 이미 오래된 방식

딥러닝 개발자의 축복이자 저주 중 하나는 신경망 아키텍처가 매우 빠르게 진화한다는 점이다. 그렇다고 이 장에서 우리가 살펴본 내용이 더 이상 필요 없을 정도로 오래됐다는 뜻은 아니다. 최신의 훌륭한 아키텍처를 빈틈없이 설명하는 건 다른 책의 역할로 미뤄두자(아마 그 책을 쓰는 이들도 완벽한 최신 내용을 저술하는 건 금세 포기할 것이다). 이렇게 빠른 변화로부터 얻어야 하는 교훈은 논문에 담겨 있는 수식을 능숙하게 실제 파이토치 코드로 작성하거나, 최소한 같은 의도로 다른 사람들이 작성한 코드를 이해할 줄 알아야 한다는 점이다. 이 책의 후반부 몇 개 장에서 우리는 아이디어를 실제 파이토치 모델로 구현하는 데 필요한 기본적인 기술들을 살펴보게 될 것이다.

8.6 결론

많은 작업을 거치면서 드디어 우리의 상상 속 친구 제인의 블로그를 위해 이미지를 필터링하는 모델을 만들었다. 이제 들어오는 이미지를 자르고 크기를 32×32 로 바꾼 후 모델이 무엇인지 확인하면 된다. 사실 우리는 문제의 일부만을 푼 것이지만 이조차도 긴 여정이었다.

방금 문제의 일부만을 풀었다고 말한 것은, 아직 알지 못하지만 조만간 마주해야 할 재미있는 부분들이 남아 있다는 의미다. 예를 들어 더 큰 이미지로부터 새와 비행기를 구별하는 것을 생각해볼 수 있다. 이미지에서 특정 물체 주위를 둘러싸는 박스를 그리는 일은 현재 우리의 모델이 할 수 없는 일이다.

다른 문제로는, 카메라 앞으로 고양이 프레드가 걸어가는 경우다. 우리의 모델은 주저없이 새 모양의 고양이에 대한 의견을 제시할 것이다! 아마도 0.99의 확률로 ‘비행기’나 ‘새’를 출력할 것이다. 훈련 분포와 거리가 꽤 먼 샘플에 대해 높은 신뢰도로 추정해버리는 문제를 **과잉 일반화 overgeneralization**라고 부르는데, 이는 좋은 성능의 모델을 제품화한 후 마주치게 되는 주요 문제 중의 하나로서 입력을 신뢰할 수 없는 경우에 발생한다(실세계에서 일어나는 대부분의 문제가 이에 해당한다).

이 장에서 우리는 실제로 동작하는, 꽤 괜찮은 수준의 모델을 파이토치로 만들어 이미지로부터 학습했다. 이를 통해 우리는 컨볼루션 신경망에 대한 직관적인 이해를 넓혔다. 또 모델을 더 넓고 깊게 만드는 법도 알아봤으며, 과적합 같은 영향도 통제해봤다. 아직은 수박 겉핥기에 그친 수준이지만 7장과 비교하면 한 단계 더 발전했다. 이제 우리는 딥러닝 프로젝트를 수행하면서 부딪힐 도전들을 마주할 탄탄한 기초도 닦았다.

이제 파이토치 기본 사용법과 일반 기능에 익숙해졌으므로 좀 더 큰 문제를 다룰 준비가 됐다. 지금까지는 소소한 문제들을 한 개 장이나 두 개 장에 걸쳐 살펴봤다. 이제는 여러 개 장을 거쳐 더 큰 실세계의 문제에 도전해보자. 2부에서는 폐암을 자동으로 진단하는 문제를 다룰 예정인데, 파이토치를 사용해 프로젝트 전반을 구현할 수 있도록 먼저 파이토치 API에 익숙해지는 단계를 거치게 된다. 다음 9장에서는 문제를 상위 레벨에서 설명하고 사용할 데이터에 대해 구체적으로 살펴볼 것이다.

8.8 핵심 요약

- 컨볼루션은 이미지를 다루는 피드포워드 신경망의 선형 연산으로 사용할 수 있다. 컨볼루션으로 더 적은 파라미터를 사용하고 지역성과 평행이동 불변성을 다룰 수 있다.
- 여러 개의 컨볼루션을 중간에 활성화 함수와 맥스 풀링을 두어 층층이 쌓으면 컨볼루션을 더 작은 피쳐 이미지에 적용하는 효과를 가지게 하므로, 결과적으로 깊이가 늘어날수록 모델이 입력 이미지의 더 넓은 영역에서 공간적인 관계를 효율적으로 처리한다.
- `nn.Module`의 서브클래스이기만 하면 자신과 모든 하위 자식의 파라미터를 얻어올 수 있다. 이 방법으로 파라미터 수를 세거나 최적화 함수에 전달하거나 값을 점검할 수 있다.
- 함수형 API는 내부 상태를 저장할 필요가 없는 모듈을 제공한다. 그래서 파라미터를 유지할 필요가 없으며 훈련도 필요 없는 연산에서 사용한다.
- 모델 파라미터는 훈련되면 한 줄의 코드로 디스크에 저장하거나 디스크에서 불러올 수 있다.